Міністерство освіти і науки України
Національний університет «Полтавська політехніка імені Юрія Кондратюка»
Навчально-науковий інститут інформаційних технологій та робототехніки
Кафедра комп'ютерних та інформаційних технологій і систем

ЗВІТ

з лабораторної роботи №4
з дисципліни «Технології на платформі .NET»

на тему: «REST API на ASP.NET Core Web API, впровадження залежностей, вебзастосунок ASP.NET MVC»

Виконав: студент групи 402-ТН
Желудков Віталій Віталійович

Перевірив: Тютюнник П. Б.

Полтава — 2026

1. Опис виконаної роботи

Роботу виконано в гілці `lab4`, створеній від гілки `lab3`. Пул реквест лабораторної роботи №3 ще не закрито, тому PR створено з гілки `lab4` у `lab3`. До рішення `GameStudio.sln` додано два нові проєкти, тепер їх сім:

Проєкт	Призначення
`GameStudio.REST`	**новий:** ASP.NET Core Web API (шаблон `webapi`, контролери): REST API з CRUD-операціями над даними з БД ЛР3, Swagger
`GameStudio.MVC`	**новий (додаткове завдання):** вебзастосунок ASP.NET Core MVC зі сторінками, згенерованими вбудованим генератором ASP.NET
`GameStudio.Common`	доменна модель, інтерфейси `IRepository<T>`, `ICrudServiceAsync<T>`, асинхронний CRUD-сервіс `CrudServiceAsync<TDomain, TModel>` з ЛР3
`GameStudio.Infrastructure`	контекст `GameStudioContext`, моделі БД, міграції, репозиторій `Repository<T>`, перетворювачі (додано `ProgrammerMapper`, `ArtistMapper`)
`GameStudio.Nosql`	репозиторій MongoDB (ЛР3, додаткове завдання)
`GameStudio.Console`	консольний застосунок ЛР1–ЛР3
`GameStudio.Tests`	тести NUnit; додано 12 інтеграційних тестів REST API

1.1. Трирівнева архітектура GameStudio.REST

Застосунок розділено на три рівні. Кожен рівень залежить лише від нижчого: контролери не знають ні про EF Core, ні про моделі БД, а лише викликають інтерфейс `ICrudServiceAsync<T>`.

```
Клієнт (Swagger UI, Postman, браузер)  ── HTTP-запити, JSON
   │
   ▼
[Представлення]     GameStudio.REST
                    контролери, моделі API (папка Models), перевірка даних
   │  ICrudServiceAsync<T>
   ▼
[Бізнес-логіка]     GameStudio.Common
                    доменні моделі та їхні правила, CrudServiceAsync<TDomain, TModel>
   │  IEntityRepository<T>, IModelMapper<TDomain, TModel>
   ▼
[Доступ до даних]   GameStudio.Infrastructure
                    GameStudioContext, Repository<T>, моделі БД, мапери
   │  EF Core
   ▼
SQLite: Database/gamestudio.db (ЛР3)
```

Рівень	Що містить у проєкті
Представлення	`ProjectsController`, `BugsController`, `ProgrammersController`, `ArtistsController` (спільний базовий клас `ApiControllerBase`); моделі API в папці `Models`; `ApiModelMapper` — перетворення доменна модель ↔ модель API; атрибут перевірки `SupportedEngine`; реєстрація залежностей і Swagger у `Program.cs`

Рівень	Що містить у проєкті
Бізнес-логіка	доменні класи `GameProject`, `BugReport`, `Programmer`, `Artist`, `StorePage` з правилами (зміна статусу, позначення бага виправленим, розрахунок зарплати); `ICrudServiceAsync<T>` і `CrudServiceAsync<TDomain, TModel>`
Доступ до даних	`GameStudioContext`, моделі БД `GameProjectModel`, `BugReportModel`, `ProgrammerModel`, `ArtistModel`…, конфігурації Fluent API, міграції, `Repository<T>`, мапери

1.2. Моделі API (папка GameStudio.REST/Models)

Для кожної сутності, що зберігається в БД, створено окремі моделі API. Модель містить лише ті властивості, які потрібні конкретному методу. Тому для запитів і відповідей, а також для створення та оновлення, використано різні моделі:

Модель	Де використовується	Особливості
`ProjectSummaryModel`	GET /api/projects (список)	без сторінки в магазині й команди
`ProjectModel`	GET /api/projects/{id}, відповіді POST/PUT	+ обчислені `Remaining`, `TeamSize`, вкладена `StorePage`
`ProjectCreateModel`	тіло POST /api/projects	без `Id`, `Status`, `Spent`: їх задає сервер
`ProjectUpdateModel`	тіло PUT /api/projects/{id}	як створення, + `Status`
`StorePageModel`	GET/PUT …/store-page	без ідентифікатора (зв'язок 1:1 з проєктом)
`TeamMemberModel`	GET …/team	Id, ПІБ, тип (Programmer/Artist), посада
`BugModel`	відповіді /api/bugs	усі поля бага
`BugCreateModel`	тіло POST /api/bugs	+ `ProjectId`, без `IsFixed` і дати
`BugUpdateModel`	тіло PUT /api/bugs/{id}	без `ProjectId` (проєкт не змінюється), + `IsFixed`
`ProgrammerModel` / `ArtistModel`	відповіді /api/programmers, /api/artists	+ обчислені посада, стаж, місячна зарплата
`ProgrammerRequestModel` / `ArtistRequestModel`	тіла POST і PUT	без `Id` та обчислюваних полів

Перевірка вхідних даних описана атрибутами DataAnnotations (`[Required]`, `[StringLength]`, `[Range]`, `[EnumDataType]`, `[Url]`) з повідомленнями українською. Атрибут `[SupportedEngine]` перевіряє рушій за списком доменної моделі. Переліки передаються в JSON рядками (`"status": "Released"`).

1.3. REST-ресурси, HTTP-методи та коди відповідей

Реалізовано контролери для чотирьох сутностей (вимога — не менше двох) і вкладені ресурси для всіх типів зв'язків з ЛР3. Назви ресурсів — іменники в множині, дії визначає HTTP-метод, ідентифікатор — Guid у шляху:

Метод	URI	Дія	Коди відповіді
GET	/api/projects?page=1&amount=10	список (пагінація)	200, 400
GET	/api/projects/{id}	один проєкт	200, 404
POST	/api/projects	створення	201 + Location, 400
PUT	/api/projects/{id}	оновлення	200, 400, 404, 409
DELETE	/api/projects/{id}	видалення (каскадно)	204, 404
GET / PUT / DELETE	/api/projects/{id}/store-page	сторінка в магазині (1:1)	200 / 201 або 200 / 204; 404
GET	/api/projects/{id}/bugs	баги проєкту (1:N)	200, 404
GET	/api/projects/{id}/team	команда (N:M)	200, 404
PUT / DELETE	/api/projects/{id}/team/{employeeId}	додати / вилучити учасника	204, 404
GET, POST	/api/bugs	список, створення	200 / 201, 400
GET, PUT, DELETE	/api/bugs/{id}	читання, оновлення, видалення	200 / 200 / 204, 400, 404
GET, POST	/api/programmers, /api/artists	список, прийом на роботу	200 / 201, 400
GET, PUT, DELETE	/api/programmers/{id}, /api/artists/{id}	читання, оновлення, звільнення	200 / 200 / 204, 400, 404

Усі помилки повертаються у стандартному форматі ProblemDetails (`application/problem+json`, RFC 9457) з полями `type`, `title`, `status`, `detail`. Для 400 є ще словник `errors` з помилками окремих полів. Назви полів у ньому — такі самі, як у JSON (camelCase).

1.4. Впровадження залежностей (Dependency Injection)

Контролери отримують CRUD-сервіси через конструктор, наприклад `ProjectsController(ICrudServiceAsync<GameProject> projects, ICrudServiceAsync<BugReport> bugs, ICrudServiceAsync<Employee> employees)`. Реалізації контексту, репозиторіїв, перетворювачів і CRUD-сервісу з ЛР3 зареєстровано у вбудованому контейнері ASP.NET Core (`Program.cs`, `Db` — псевдонім простору імен моделей БД):

```
// контекст БД — Scoped: окремий екземпляр на кожен HTTP-запит
builder.Services.AddDbContext<GameStudioContext>(options => options.UseSqlite(connectionString));

// репозиторій — відкритий узагальнений тип: Repository<T> для будь-якої моделі БД
builder.Services.AddScoped(typeof(IRepository<>), typeof(Repository<>));
builder.Services.AddScoped(typeof(IEntityRepository<>), typeof(Repository<>));

// перетворювачі «доменна модель ↔ модель БД»
```

```
builder.Services.AddScoped<IModelMapper<GameProject, Db.GameProjectModel>, GameProjectMapper>();
builder.Services.AddScoped<IModelMapper<BugReport, Db.BugReportModel>, BugReportMapper>();
builder.Services.AddScoped<IModelMapper<Employee, Db.EmployeeModel>, EmployeeMapper>();
builder.Services.AddScoped<IModelMapper<Programmer, Db.ProgrammerModel>, ProgrammerMapper>();
builder.Services.AddScoped<IModelMapper<Artist, Db.ArtistModel>, ArtistMapper>();

// ---------- Рівень бізнес-логіки (GameStudio.Common): асинхронний CRUD-сервіс з ЛР3 ----------
builder.Services.AddScoped<ICrudServiceAsync<GameProject>, CrudServiceAsync<GameProject,
Db.GameProjectModel>>();
builder.Services.AddScoped<ICrudServiceAsync<BugReport>, CrudServiceAsync<BugReport,
Db.BugReportModel>>();
builder.Services.AddScoped<ICrudServiceAsync<Employee>, CrudServiceAsync<Employee, Db.EmployeeModel>>();
builder.Services.AddScoped<ICrudServiceAsync<Programmer>, CrudServiceAsync<Programmer,
Db.ProgrammerModel>>();
builder.Services.AddScoped<ICrudServiceAsync<Artist>, CrudServiceAsync<Artist, Db.ArtistModel>>();
```

Усі залежності мають час життя Scoped: на кожен HTTP-запит створюється один `GameStudioContext`, і його спільно використовують репозиторії, мапери та сервіси цього запиту. Рядок підключення береться з `appsettings.json` (`ConnectionStrings:GameStudio`). Якщо він порожній, використовується файл БД з ЛР3.

1.5. Інтеграційні тести API

Додано 12 тестів (`RestApiTests`). `WebApplicationFactory<Program>` запускає застосунок у пам'яті з окремим тимчасовим файлом SQLite, а `HttpClient` надсилає справжні HTTP-запити. Тести перевіряють коди відповідей, заголовок `Location`, формат помилок, пагінацію, каскадне видалення і зв'язки. Разом із тестами попередніх робіт — 61 тестів, усі пройшли (розділ 4).

1.6. Додаткове завдання: GameStudio.MVC

Вебзастосунок ASP.NET Core MVC використовує той самий `GameStudioContext` і моделі БД з ЛР3. Контролери та сторінки створення, перегляду, редагування й видалення згенеровано вбудованим генератором ASP.NET (MVC Controller with views, using Entity Framework) для проєктів, програмістів, художників і багів. Згенерований код доопрацьовано: списки вибору для переліків, підписи українською, пагінація списків, на сторінці проєкту — пов'язані дані (сторінка в магазині, команда, баги). Детальніше — у розділі 3.

Pull request (гілка `lab4` → `lab3`):
https://github.com/vitalikcraft666-design/NUPP_NET_2026_402_TK_Zheludkov_Lab/pull/4

Гілка lab4:
https://github.com/vitalikcraft666-design/NUPP_NET_2026_402_TK_Zheludkov_Lab/tree/lab4

Репозиторій:
https://github.com/vitalikcraft666-design/NUPP_NET_2026_402_TK_Zheludkov_Lab

2. Результати виконання запитів до REST API

API запущено командою `dotnet run --project GameStudio.REST` (профіль `http`, адреса `http://localhost:5160`). Запити виконано на сторінці Swagger UI (`/swagger`), яку генерує застосунок. Щоб не змінювати файл БД у репозиторії, запити виконувалися над його копією: рядок підключення задано змінною середовища `ConnectionStrings__GameStudio`.

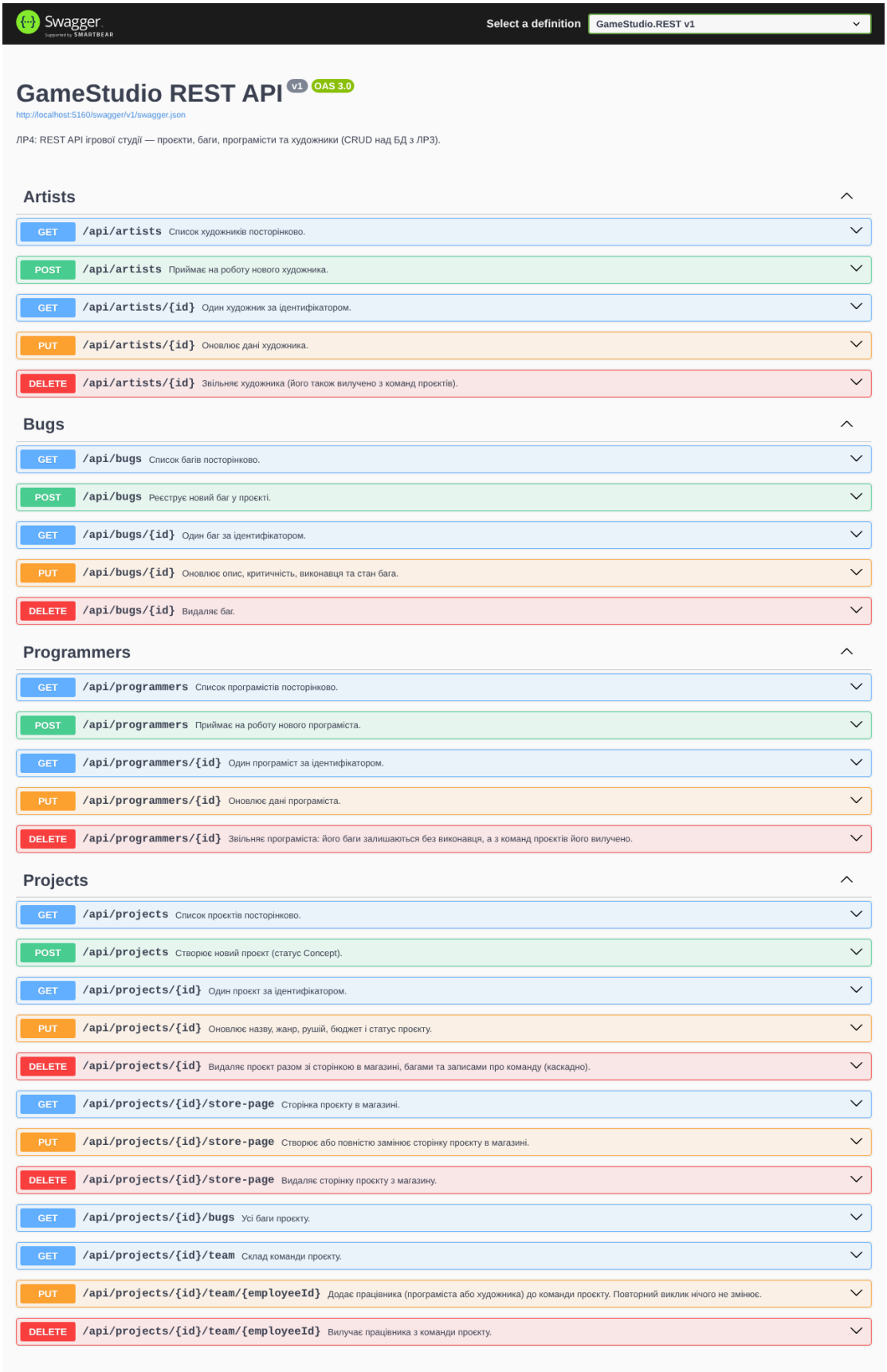


Рисунок 1 — Сторінка Swagger UI: усі операції API, згруповані за контролерами

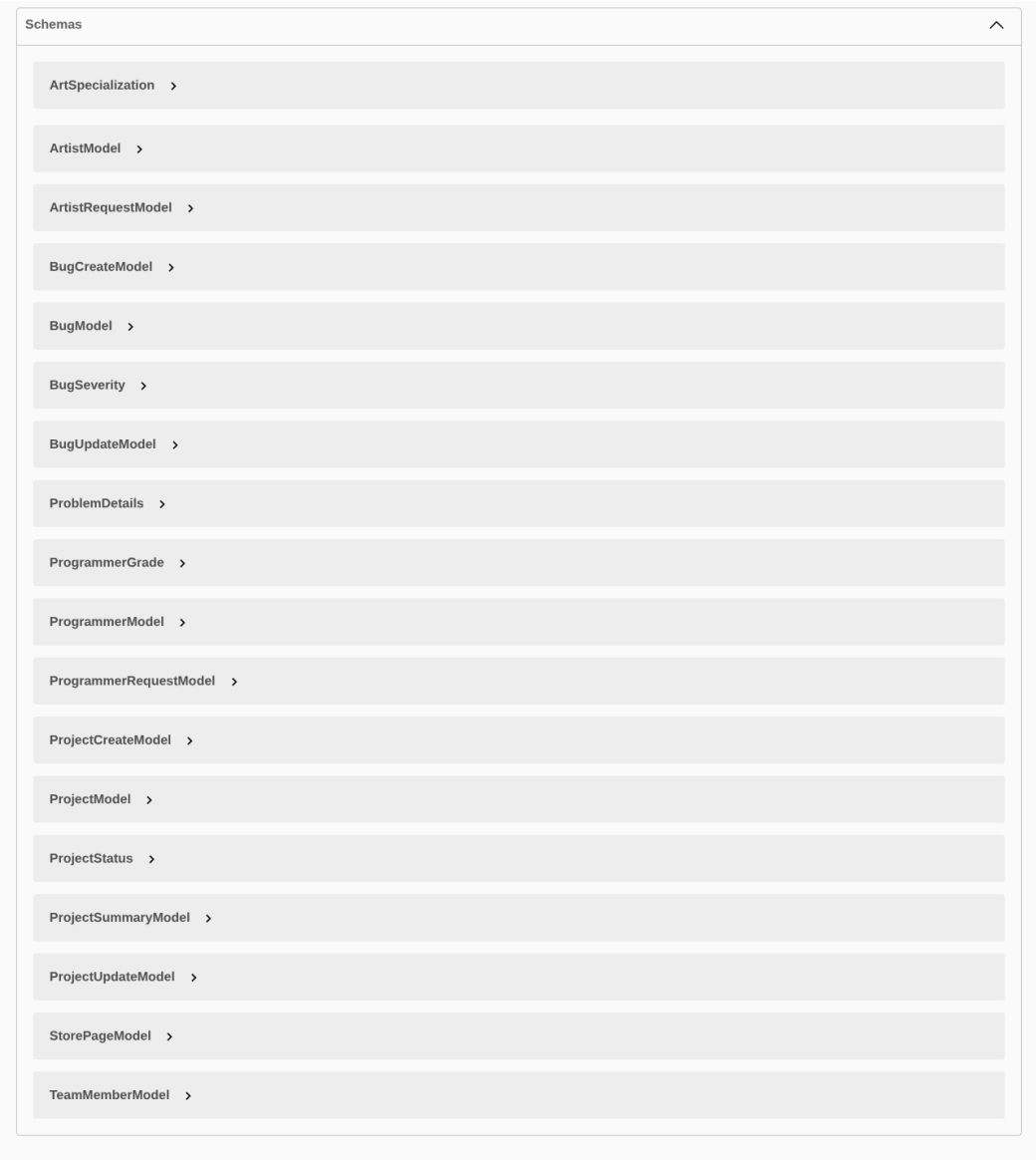


Рисунок 2 — Схеми моделей API (папка Models), згенеровані Swagger

Нижче — сценарій із 30 запитів, що послідовно перевіряє всі CRUD-операції та обробку помилок. Усі коди відповідей збіглися з очікуваними:

№	Запит	Що перевіряється	Відповідь
1	GET /api/projects	Список проєктів: сторінка 1, по 3 записи	200 OK
2	POST /api/projects	Створення проєкту; у заголовку Location — адреса нового ресурсу	201 Created
3	POST /api/projects	Некоректні дані: коротка назва, непідтримуваний рушій, нульовий бюджет — помилки по кожному полю	400 Bad Request
4	GET /api/projects/{id}	Читання створеного проєкту за Id	200 OK
5	GET /api/projects/{id}	Читання проєкту з неіснуючим Id	404 Not Found
6	PUT /api/projects/{id}	Оновлення проєкту (назва, бюджет, статус Production)	200 OK
7	PUT /api/projects/{id}/store-page	Створення сторінки проєкту в магазині (зв'язок 1:1)	201 Created
8	GET /api/projects/{id}/store-page	Читання сторінки в магазині	200 OK

№	Запит	Що перевіряється	Відповідь
9	POST /api/programmers	Прийом програміста; у відповіді — обчислені посада, стаж і зарплата	201 Created
10	GET /api/programmers	Список програмістів: сторінка 1, по 2 записи	200 OK
11	PUT /api/programmers/{id}	Підвищення програміста до Senior — зарплату перераховано	200 OK
12	GET /api/artists	Список художників: сторінка 1, по 2 записи	200 OK
13	POST /api/artists	Прийом художника	201 Created
14	PUT /api/projects/{id}/team/{employeeId}	Додавання програміста до команди проєкту (зв'язок N:M)	204 No Content
15	PUT /api/projects/{id}/team/{employeeId}	Додавання художника до команди проєкту	204 No Content
16	GET /api/projects/{id}/team	Склад команди проєкту	200 OK
17	POST /api/bugs	Реєстрація бага в проєкті	201 Created
18	POST /api/bugs	Баг з посиланням на неіснуючий проєкт	400 Bad Request
19	PUT /api/bugs/{id}	Спроба позначити баг виправленим, доки його нікому не призначено	400 Bad Request
20	PUT /api/bugs/{id}	Призначення виконавця і позначення бага виправленим	200 OK
21	GET /api/projects/{id}/bugs	Баги проєкту (зв'язок 1:N)	200 OK
22	PUT /api/programmers/{id}	Оновлення програміста з некоректними даними	400 Bad Request
23	PUT /api/projects/{id}	Випуск гри: статус Released	200 OK
24	PUT /api/projects/{id}	Спроба змінити статус уже випущеного проєкту	409 Conflict
25	DELETE /api/projects/{id}/team/{employeeId}	Вилучення художника з команди	204 No Content
26	DELETE /api/bugs/{id}	Видалення бага	204 No Content
27	DELETE /api/projects/{id}	Видалення проєкту (сторінка в магазині та записи команди видаляються каскадно)	204 No Content
28	GET /api/projects/{id}	Читання видаленого проєкту	404 Not Found
29	DELETE /api/programmers/{id}	Звільнення програміста	204 No Content
30	DELETE /api/artists/{id}	Звільнення художника	204 No Content

Запит 1. GET /api/projects → 200 OK

Список проєктів: сторінка 1, по 3 записи.

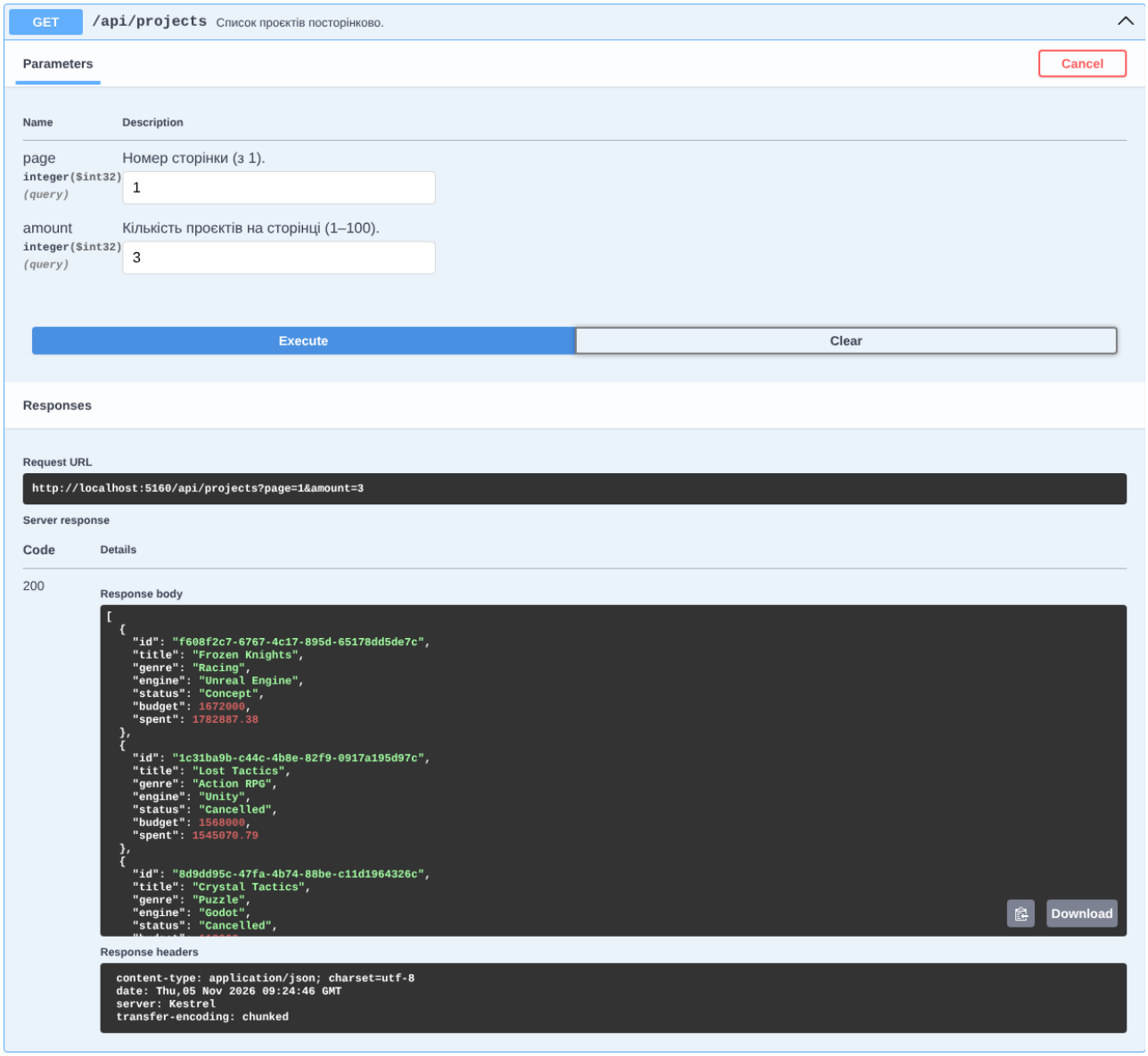


Рисунок 3 — GET /api/projects: 200 OK

Запит 2. POST /api/projects → 201 Created

Створення проєкту; у заголовку Location — адреса нового ресурсу.

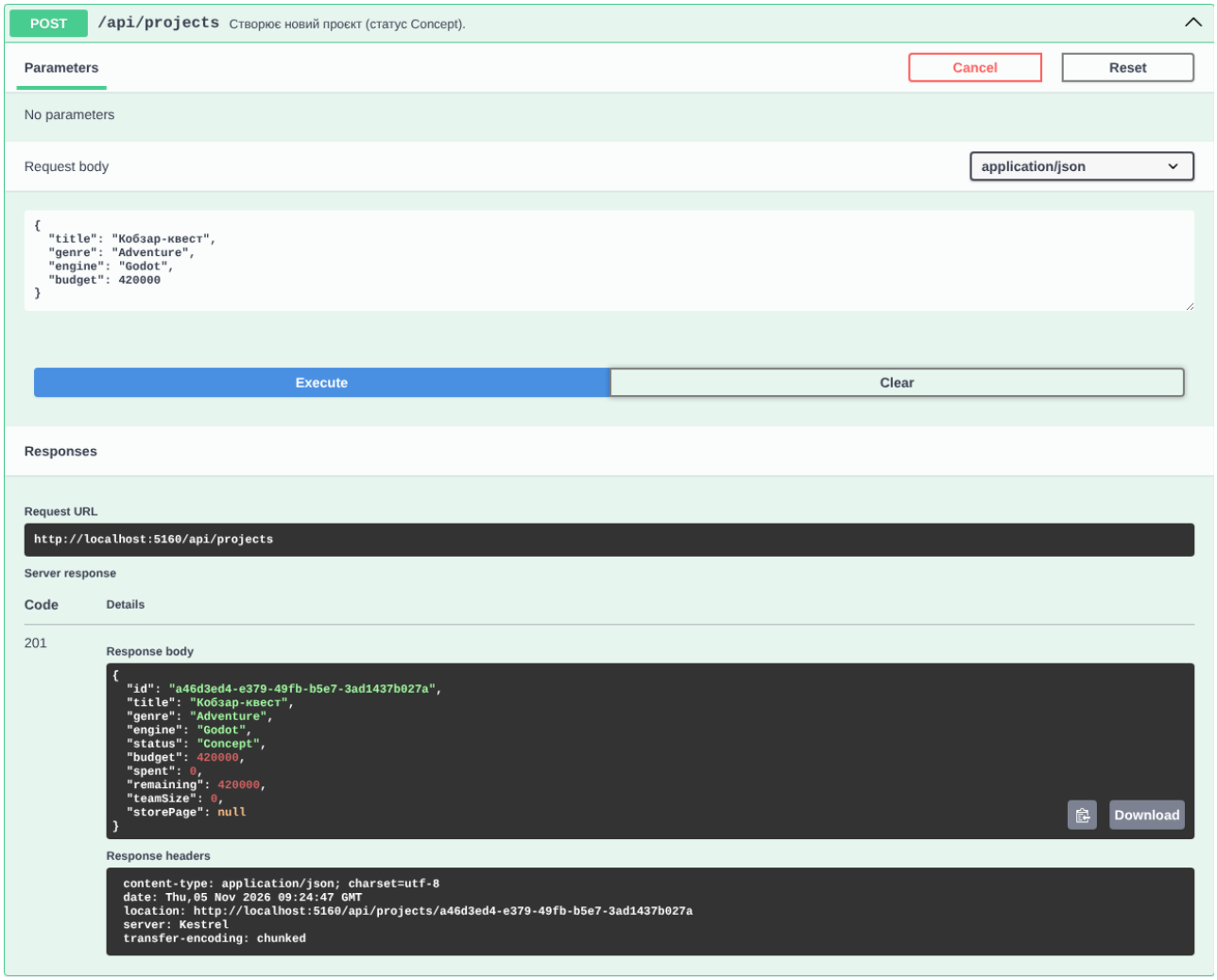


Рисунок 4 — POST /api/projects: 201 Created

Запит 3. POST /api/projects → 400 Bad Request

Некоректні дані: коротка назва, непідтримуваний рушій, нульовий бюджет — помилки по кожному полю.

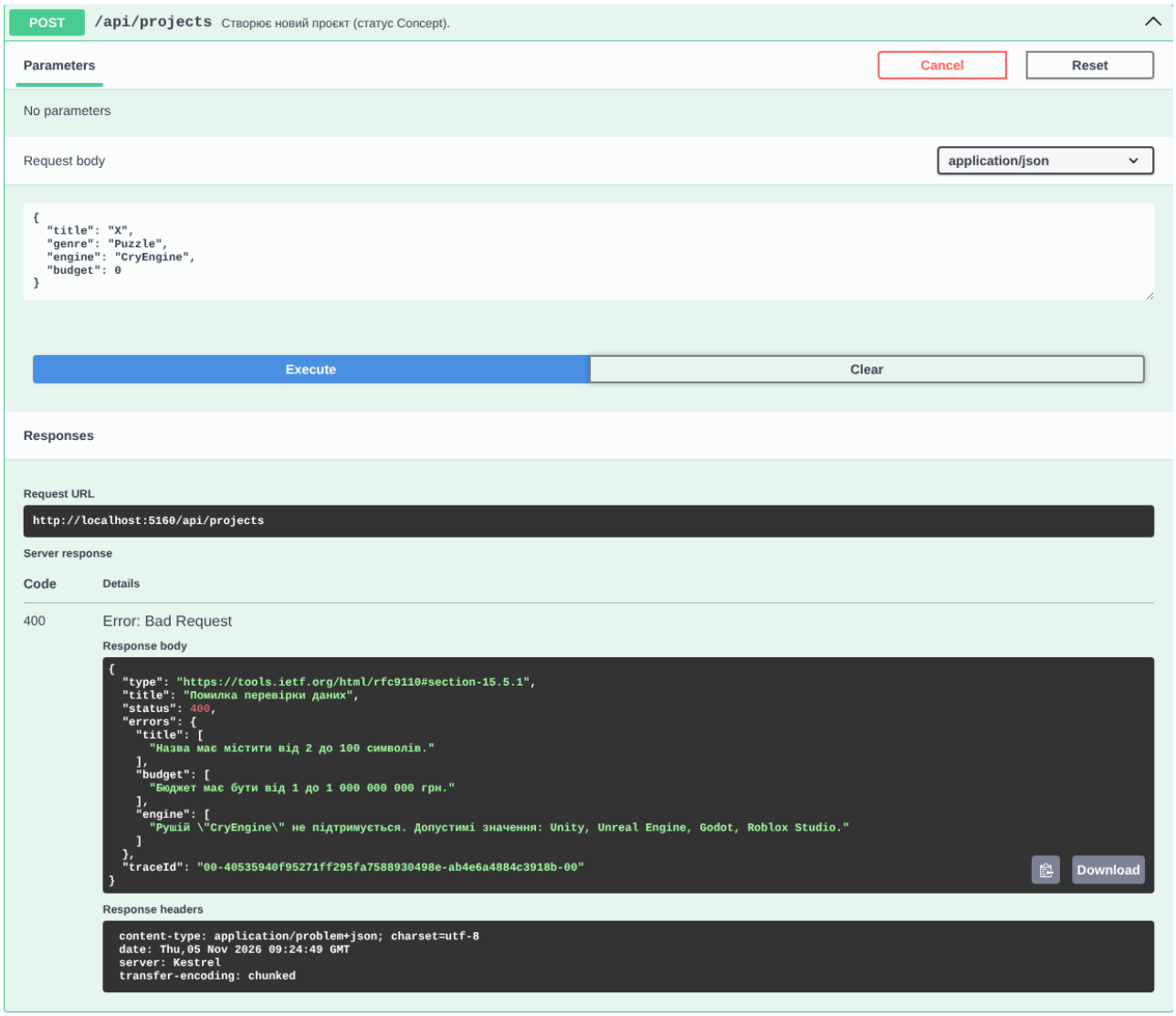


Рисунок 5 — POST /api/projects: 400 Bad Request

Запит 4. GET /api/projects/{id} → 200 OK

Читання створеного проєкту за Id.

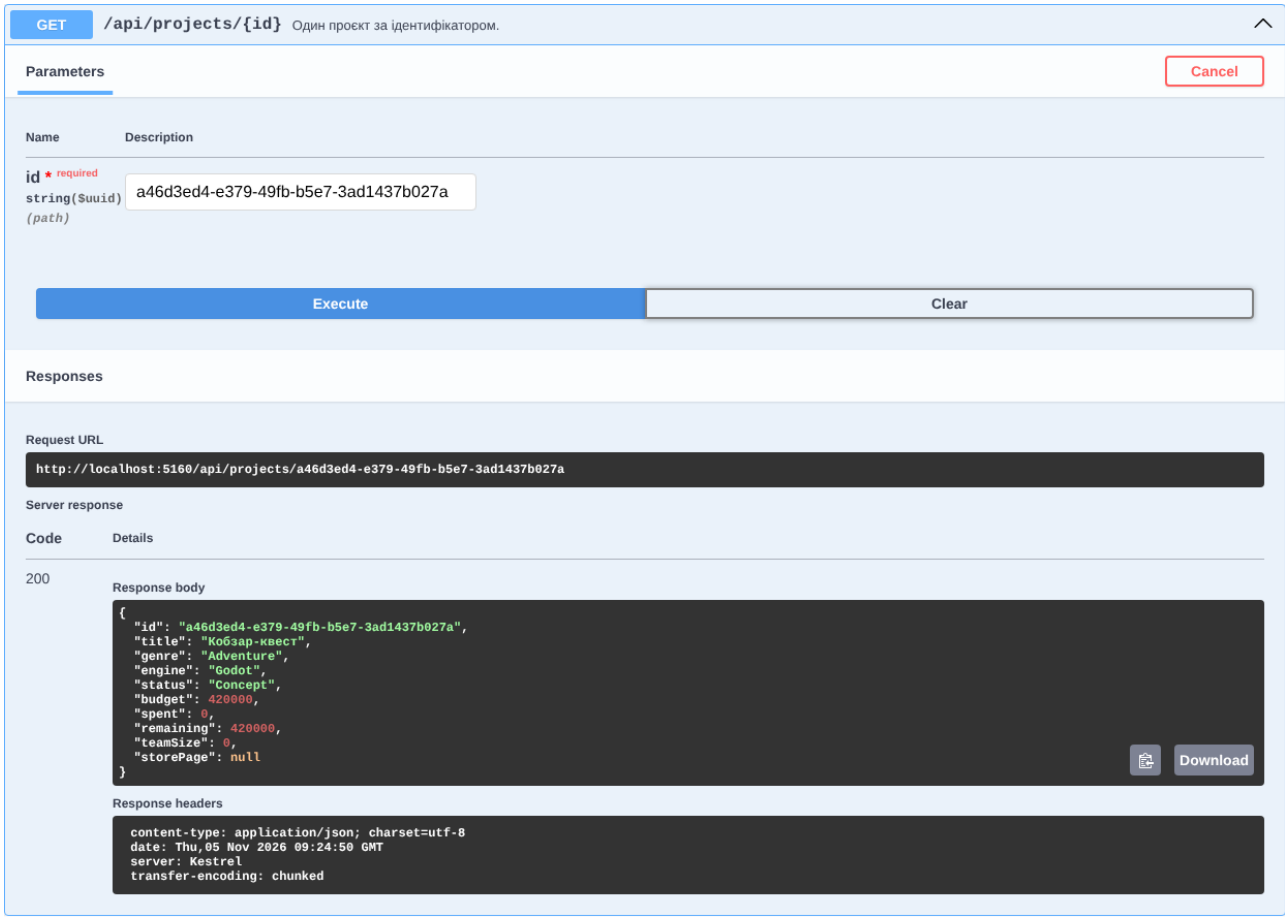


Рисунок 6 — GET /api/projects/{id}: 200 OK

Запит 5. GET /api/projects/{id} → 404 Not Found

Читання проєкту з неіснуючим Id.

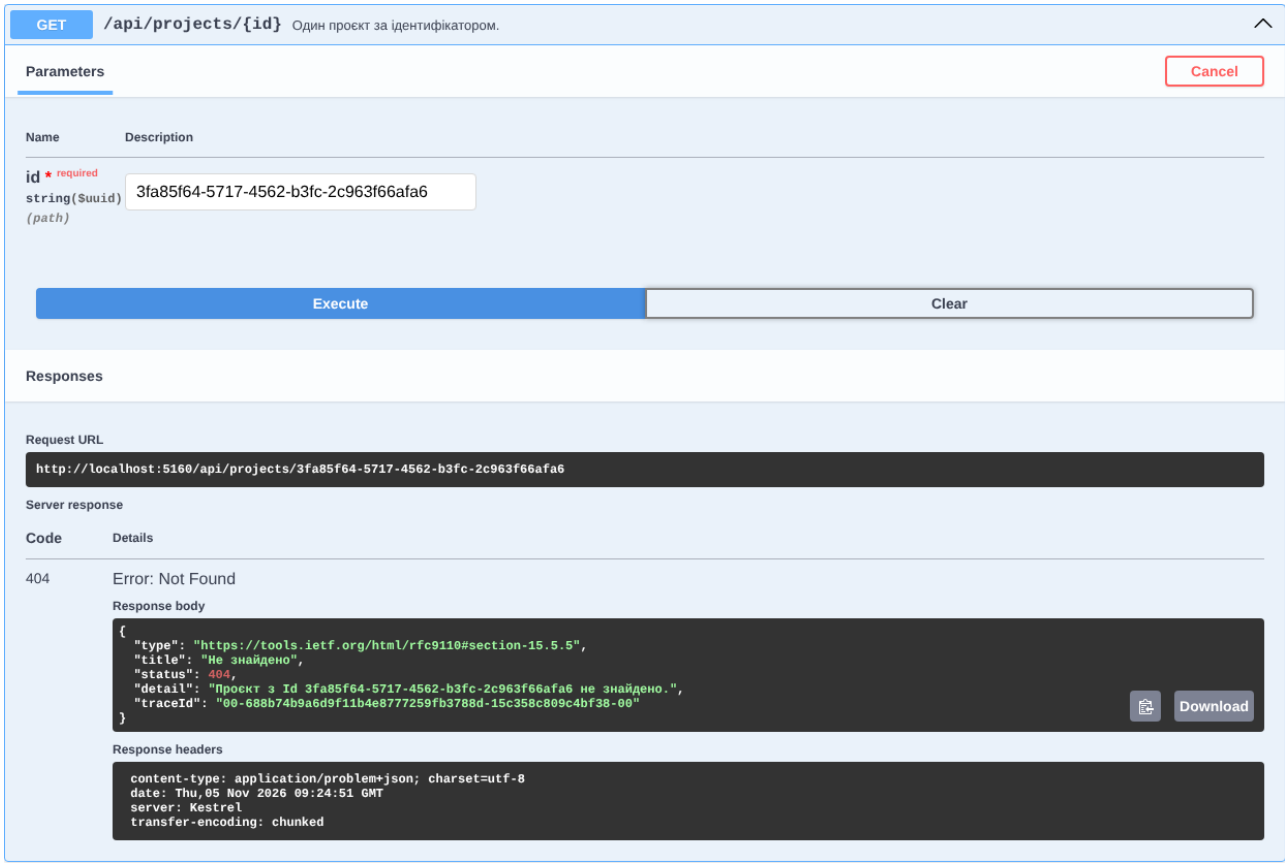


Рисунок 7 — GET /api/projects/{id}: 404 Not Found

Запит 6. PUT /api/projects/{id} → 200 OK

Оновлення проєкту (назва, бюджет, статус Production).

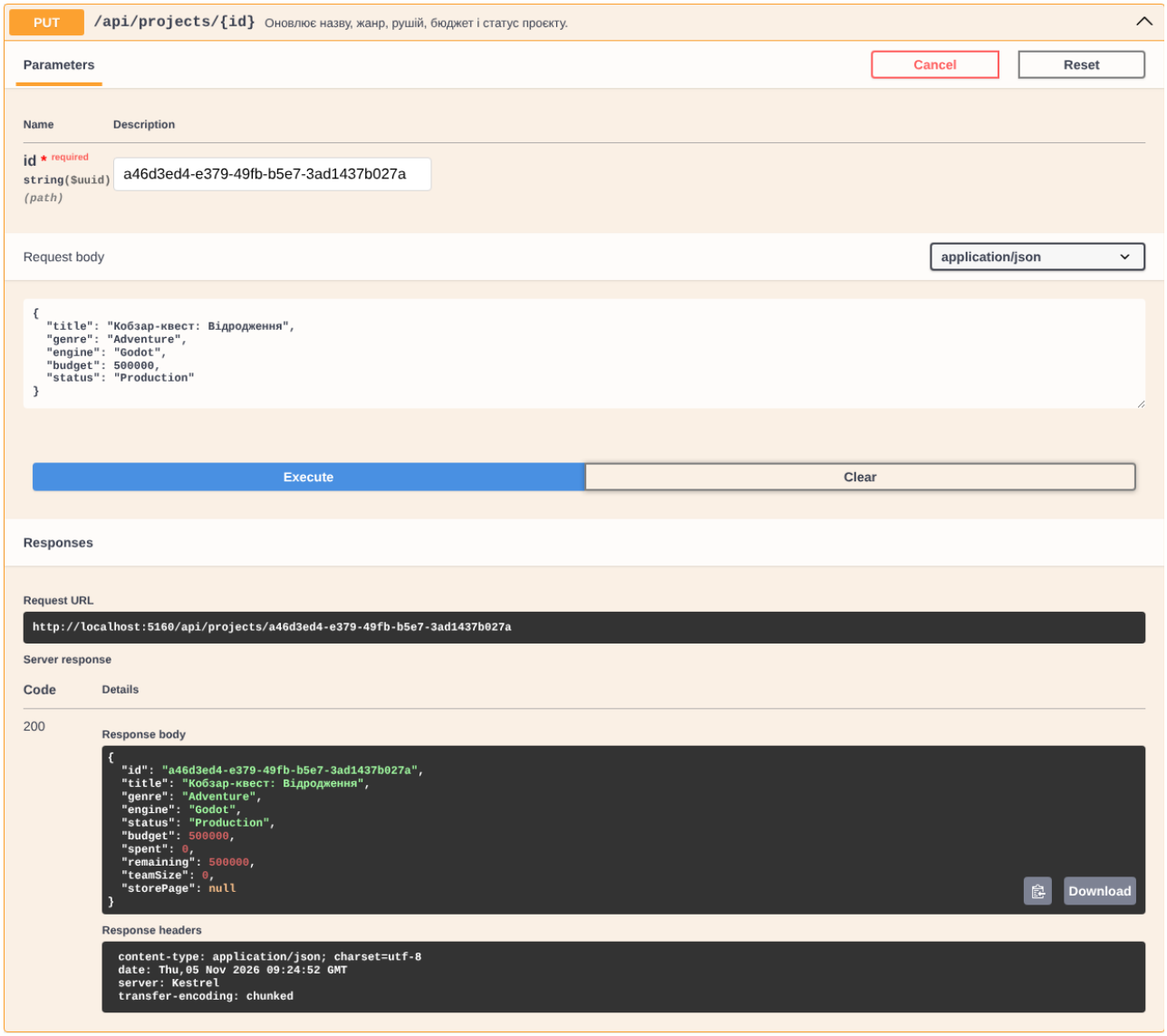


Рисунок 8 — PUT /api/projects/{id}: 200 OK

Запит 7. PUT /api/projects/{id}/store-page → 201 Created

Створення сторінки проєкту в магазині (зв'язок 1:1).

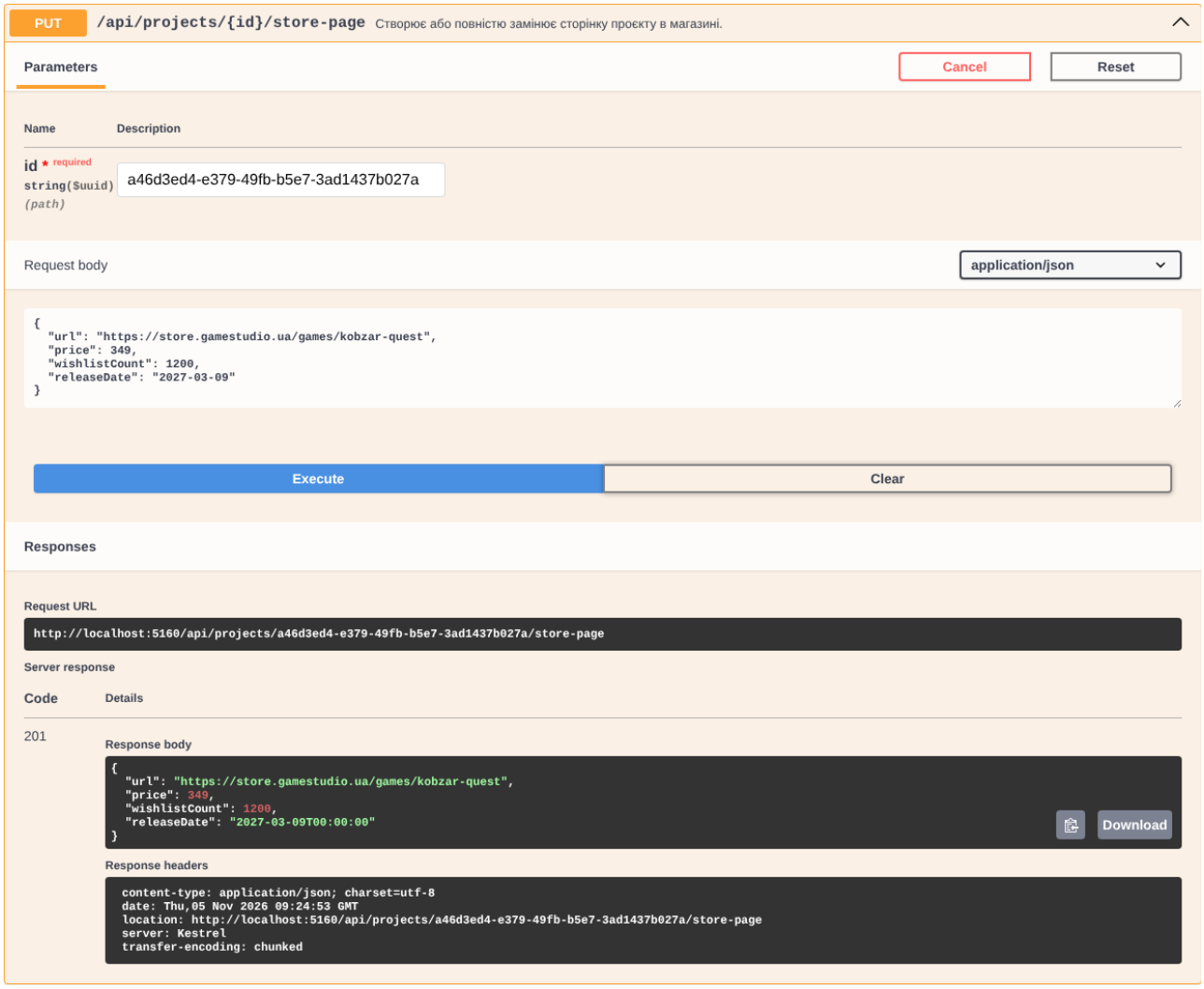


Рисунок 9 — PUT /api/projects/{id}/store-page: 201 Created

Запит 8. GET /api/projects/{id}/store-page → 200 OK

Читання сторінки в магазині.

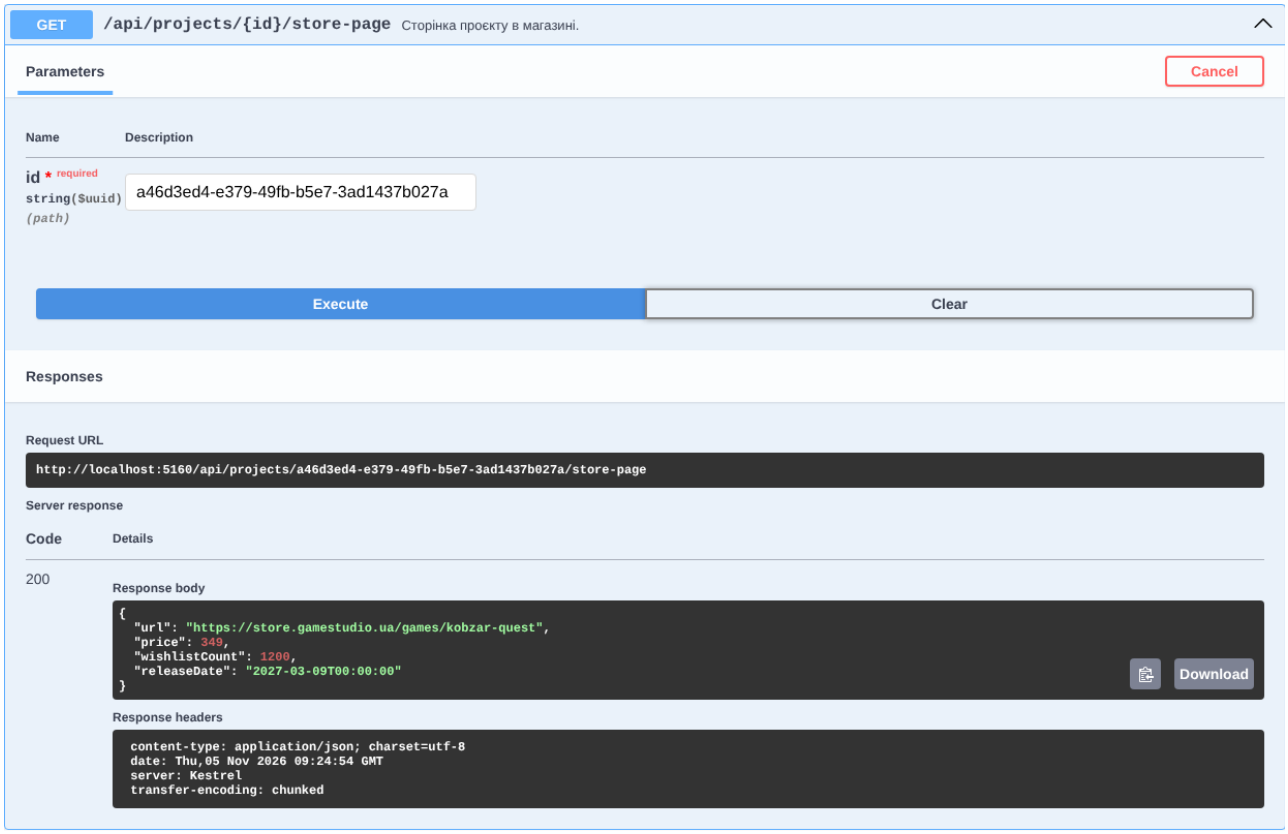


Рисунок 10 — GET /api/projects/{id}/store-page: 200 OK

Запит 9. POST /api/programmers → 201 Created

Прийом програміста; у відповіді — обчислені посада, стаж і зарплата.

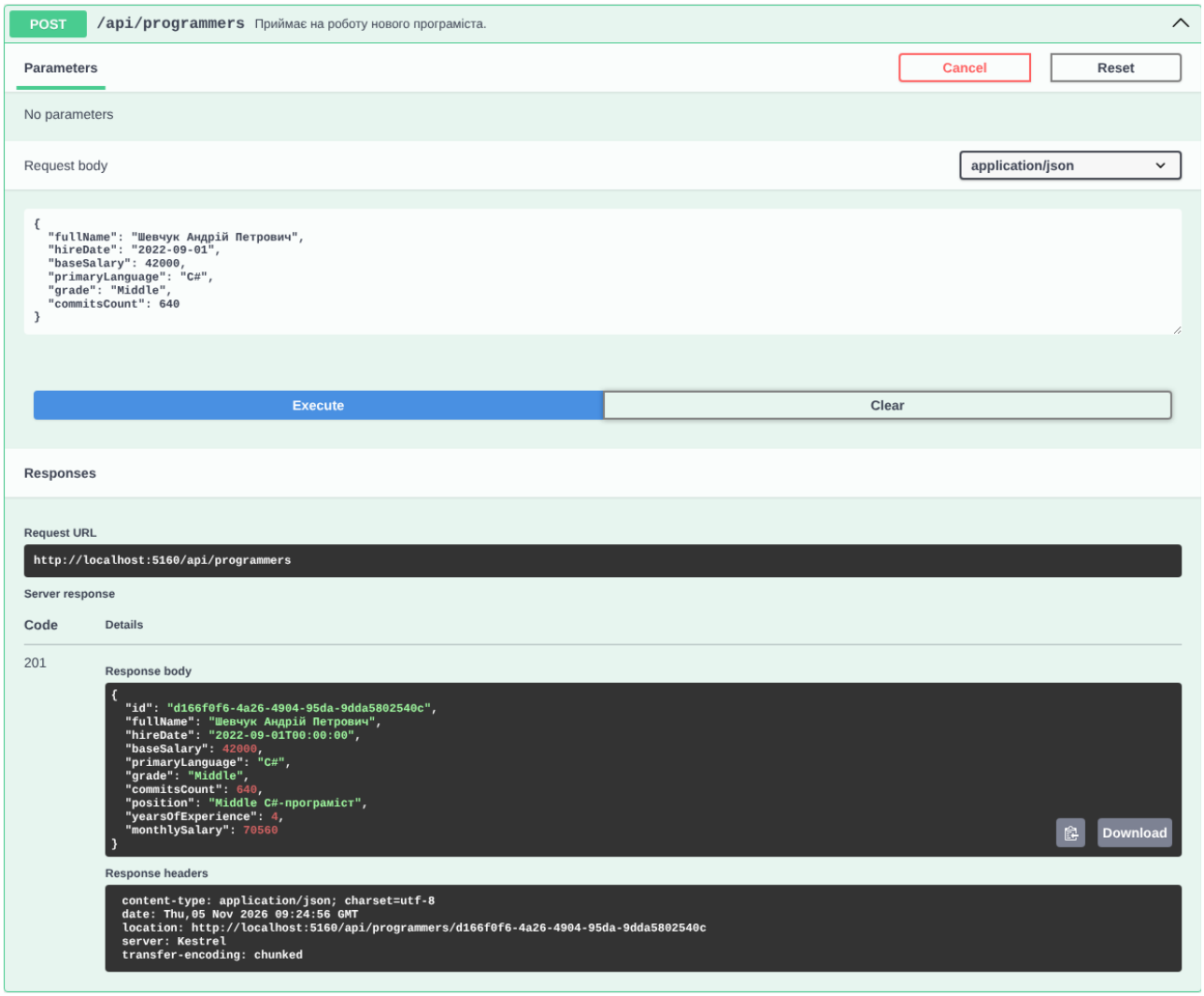


Рисунок 11 — POST /api/programmers: 201 Created

Запит 10. GET /api/programmers → 200 OK

Список програмістів: сторінка 1, по 2 записи.

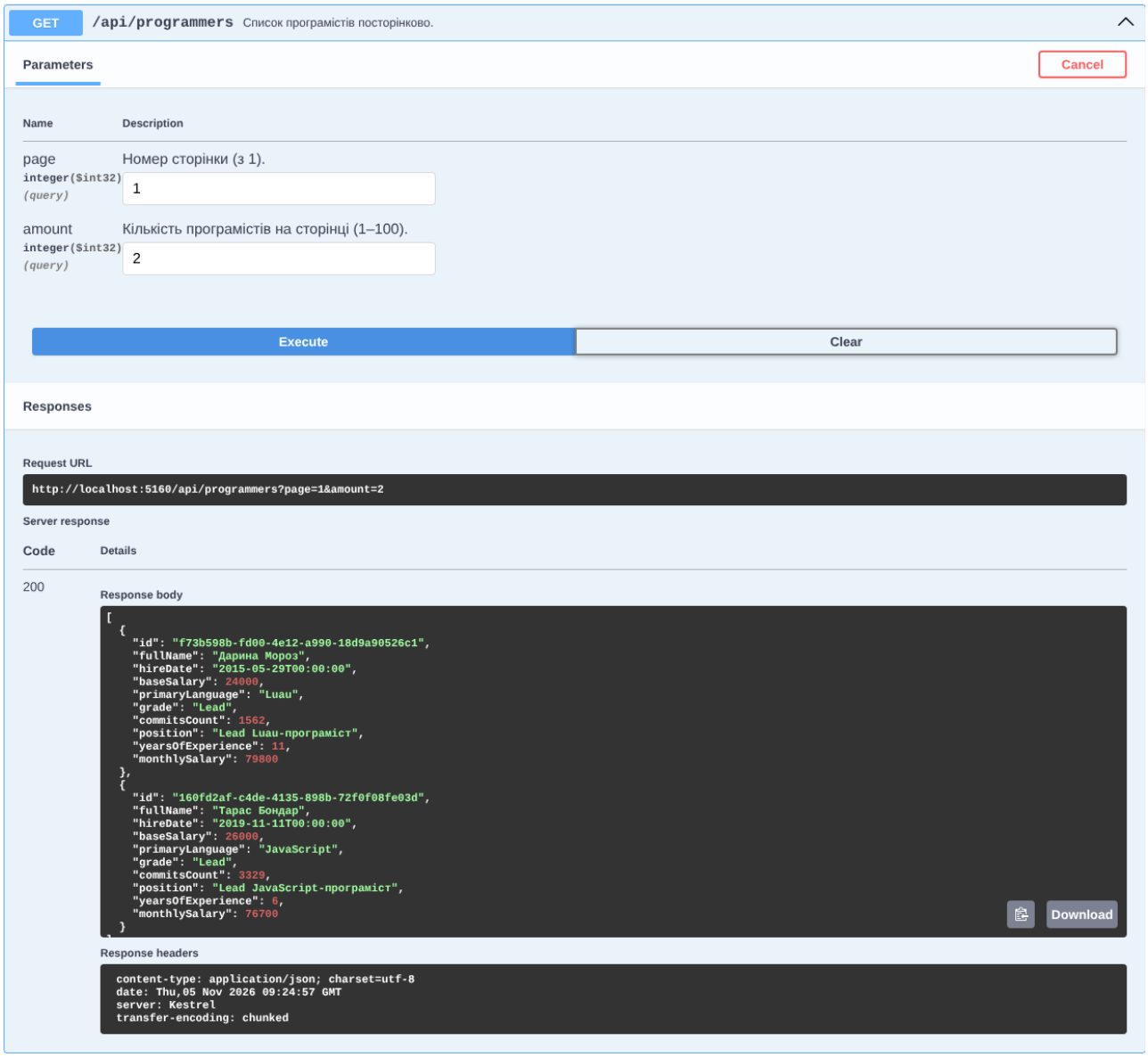


Рисунок 12 — GET /api/programmers: 200 OK

Запит 11. PUT /api/programmers/{id} → 200 OK

Підвищення програміста до Senior — зарплату перераховано.

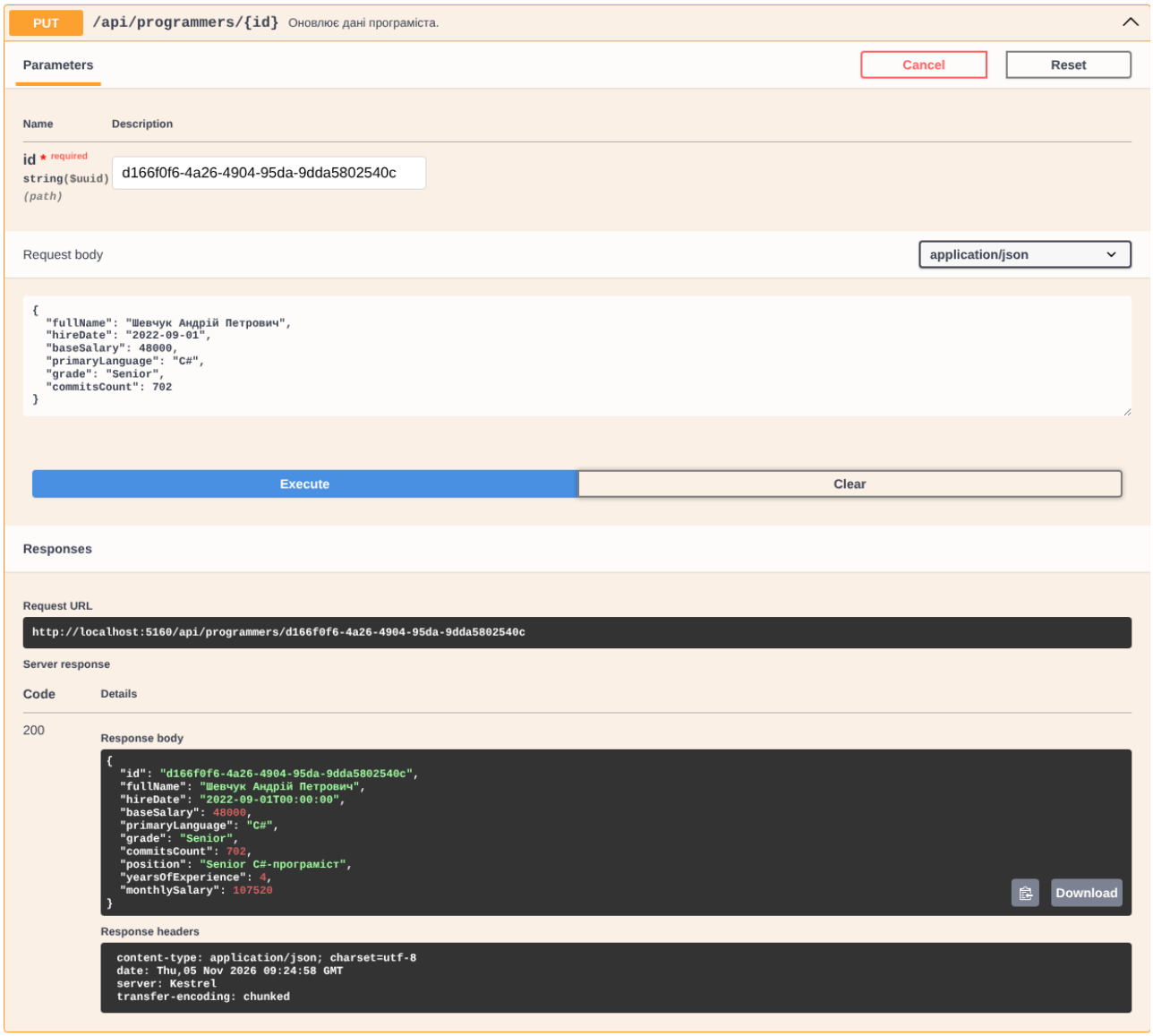


Рисунок 13 — PUT /api/programmers/{id}: 200 OK

Запит 12. GET /api/artists → 200 OK

Список художників: сторінка 1, по 2 записи.

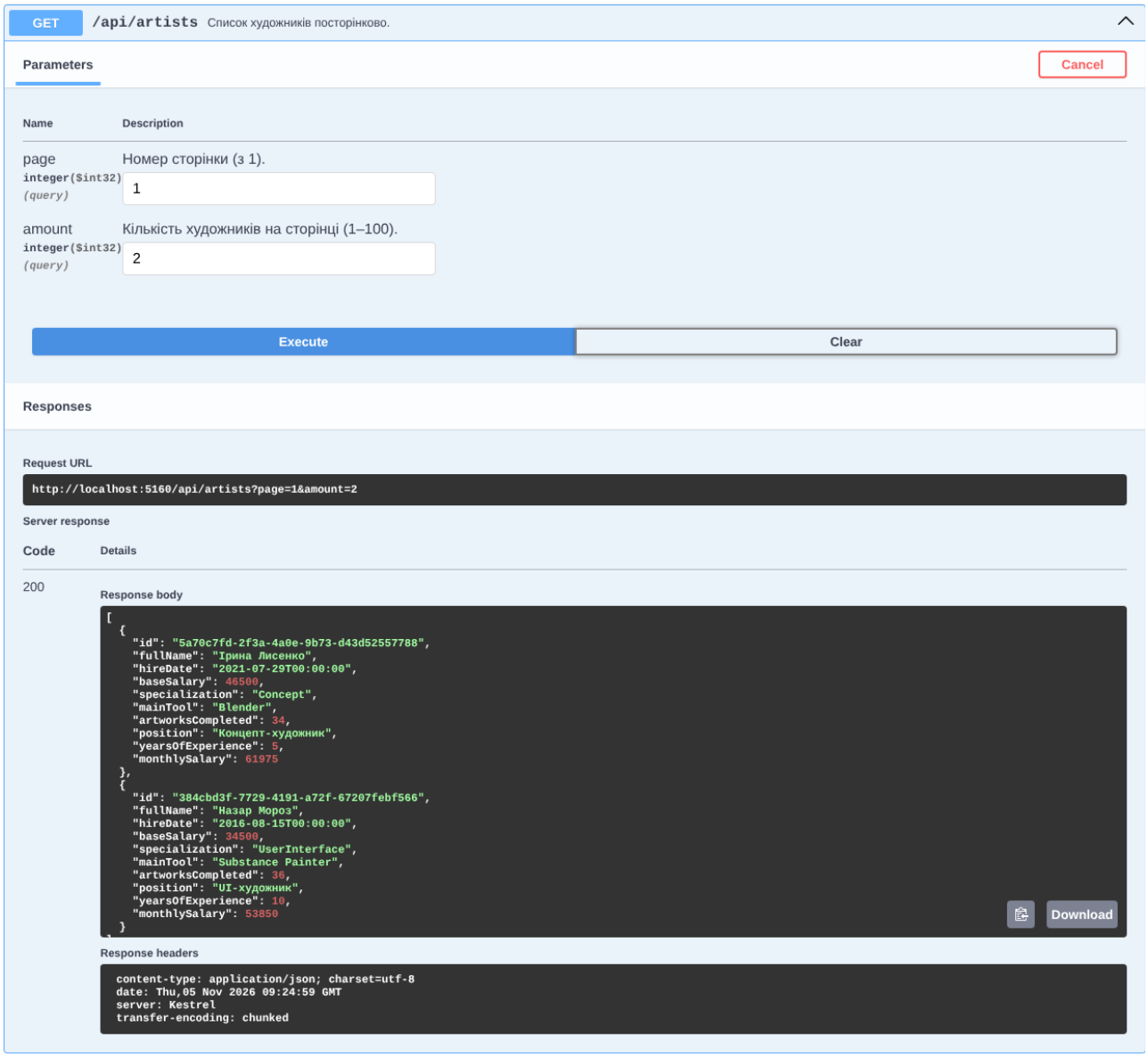


Рисунок 14 — GET /api/artists: 200 OK

Запит 13. POST /api/artists → 201 Created

Прийом художника.

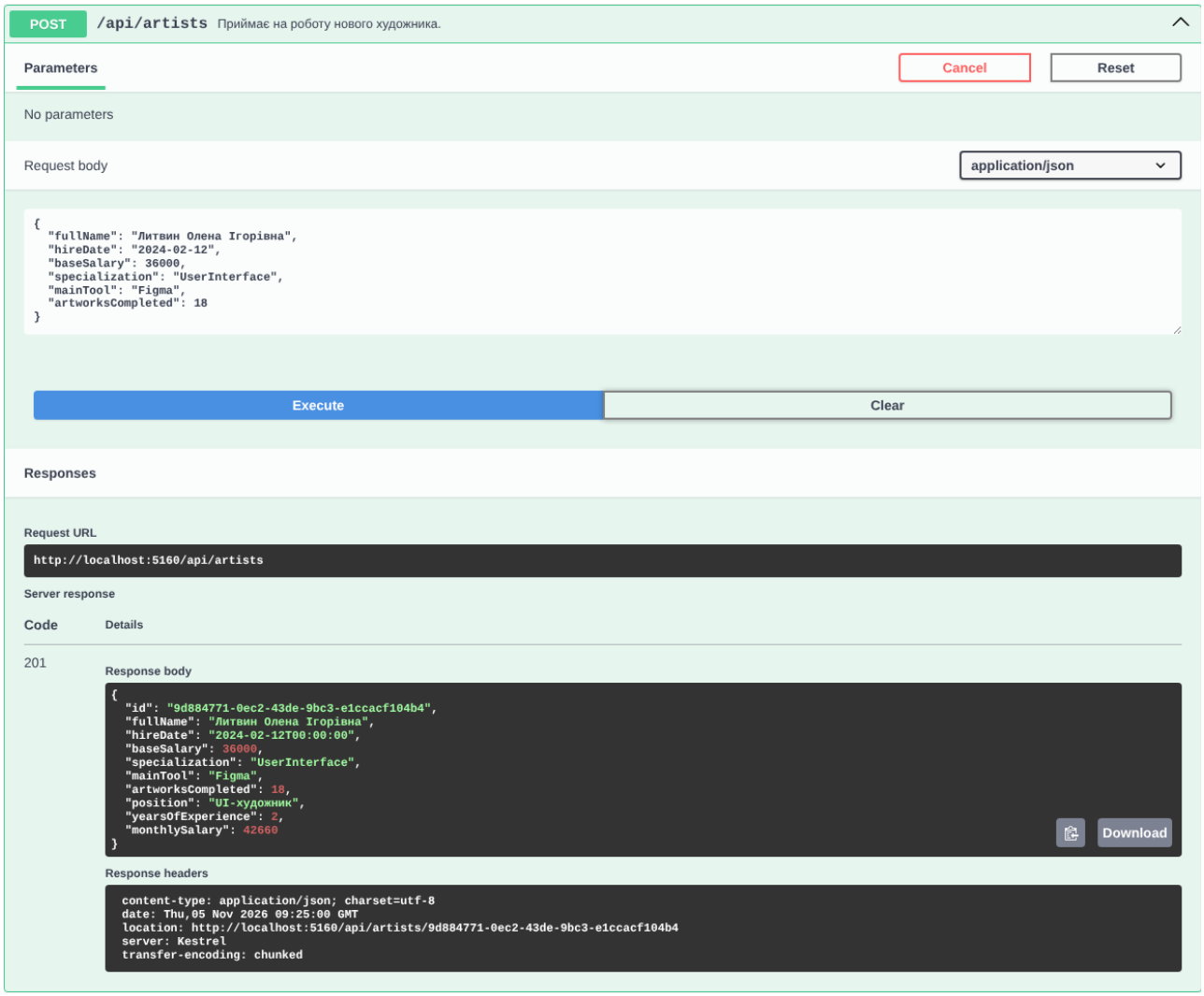


Рисунок 15 — POST /api/artists: 201 Created

Запит 14. PUT /api/projects/{id}/team/{employeeId} → 204 No Content

Додавання програміста до команди проєкту (зв'язок N:M).

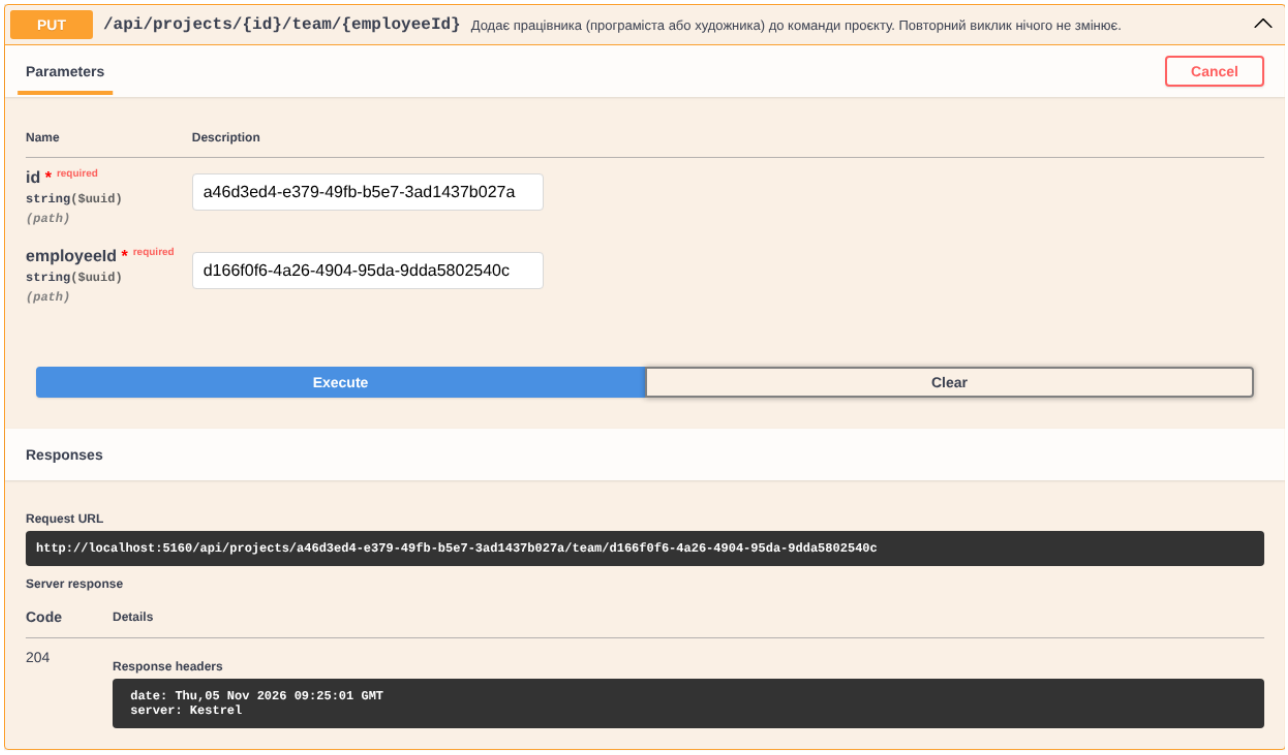


Рисунок 16 — PUT /api/projects/{id}/team/{employeeId}: 204 No Content

Запит 15. PUT /api/projects/{id}/team/{employeeId} → 204 No Content

Додавання художника до команди проєкту.

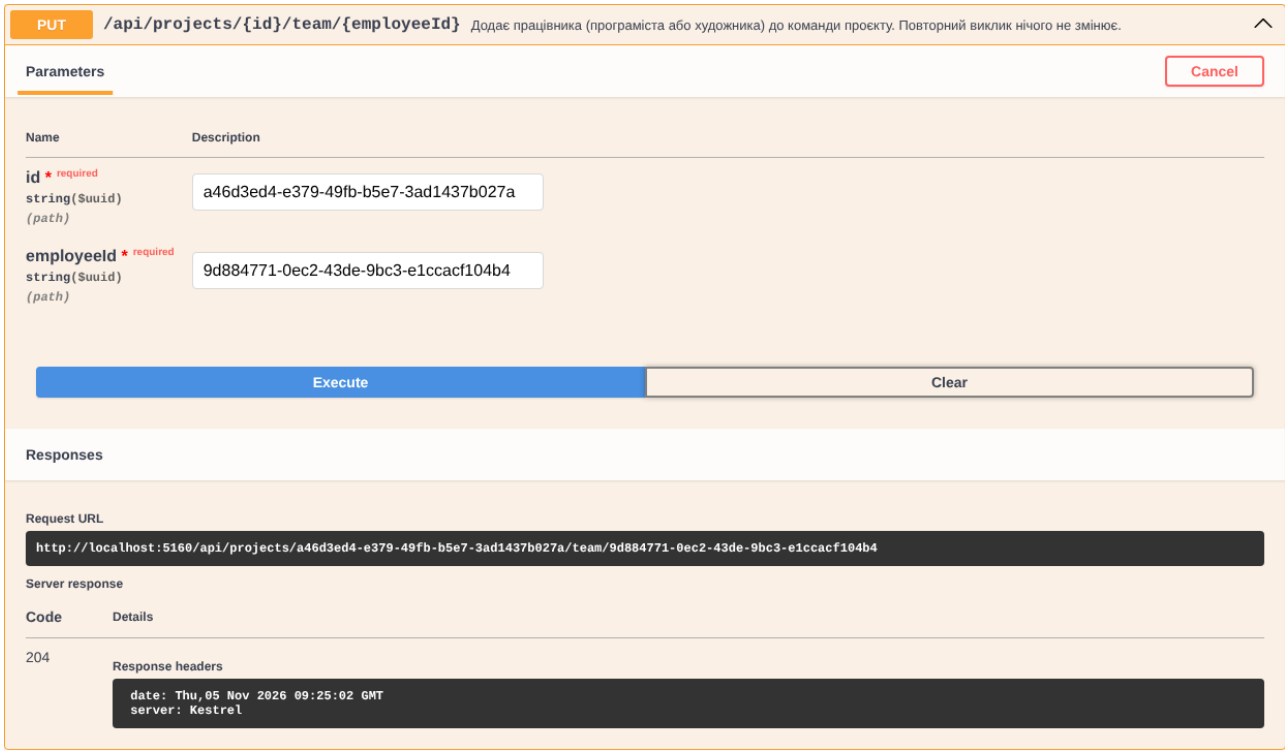


Рисунок 17 — PUT /api/projects/{id}/team/{employeeId}: 204 No Content

Запит 16. GET /api/projects/{id}/team → 200 OK

Склад команди проєкту.

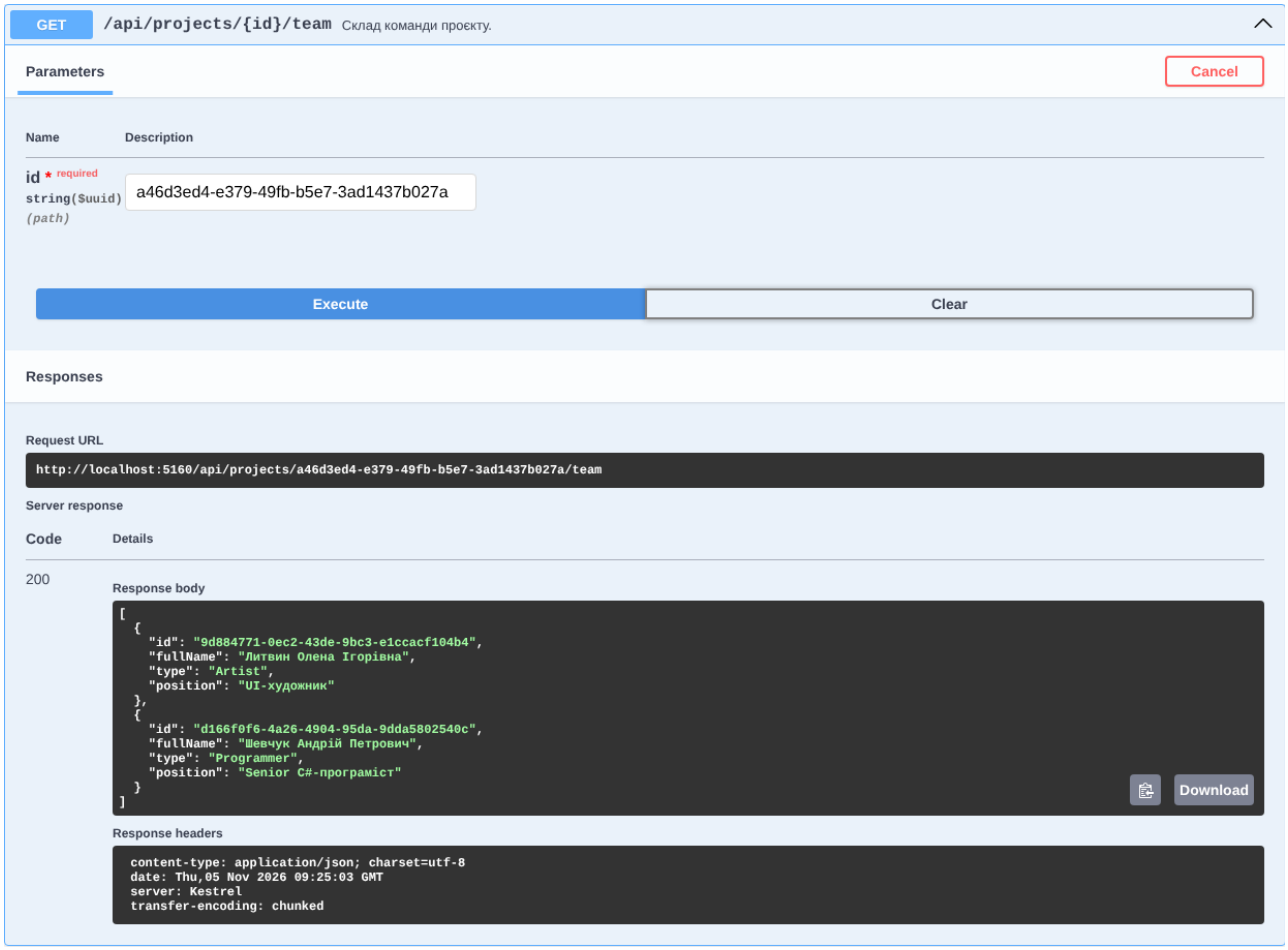


Рисунок 18 — GET /api/projects/{id}/team: 200 OK

Запит 17. POST /api/bugs → 201 Created

Реєстрація бага в проєкті.

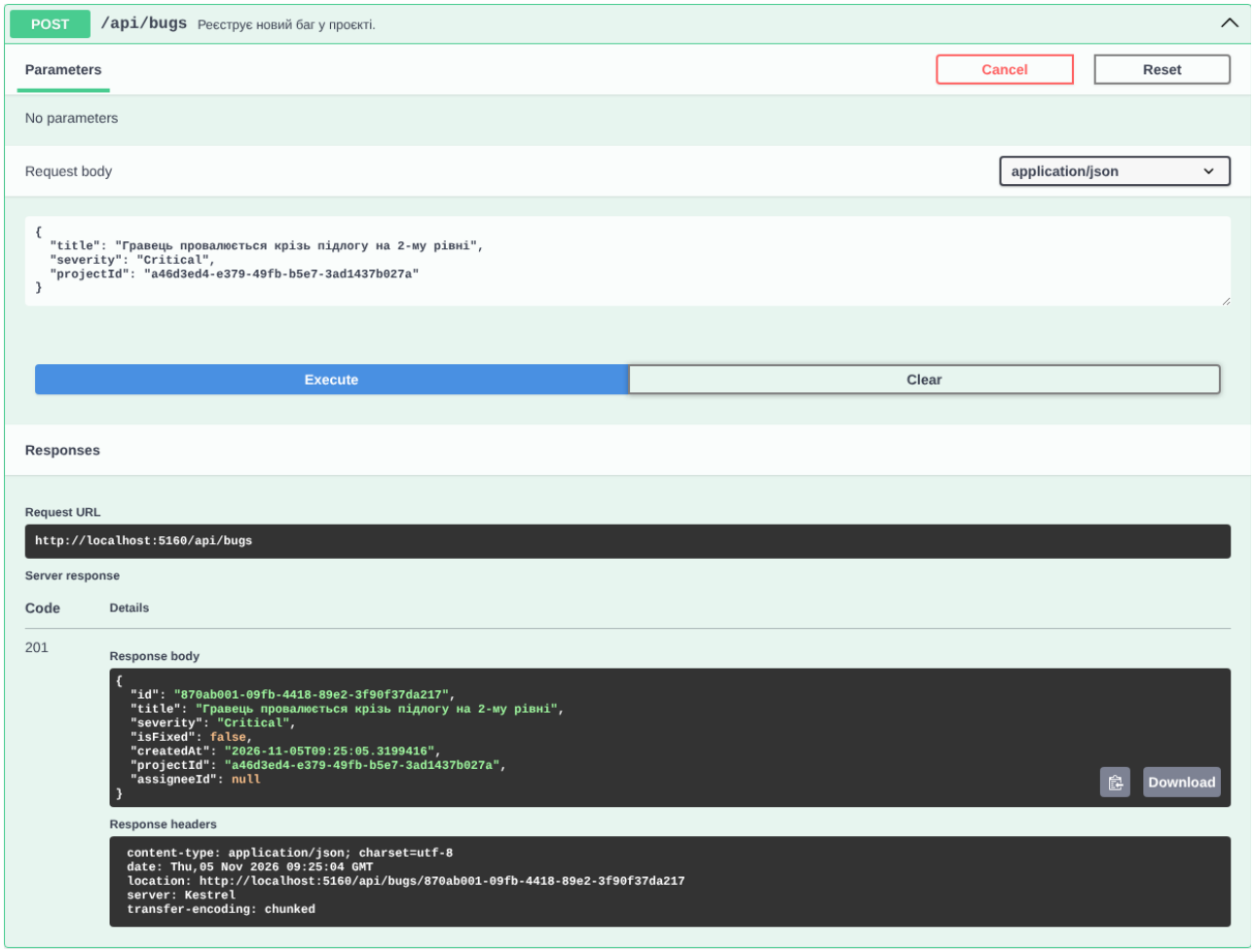


Рисунок 19 — POST /api/bugs: 201 Created

Запит 18. POST /api/bugs → 400 Bad Request

Баг з посиланням на неіснуючий проєкт.

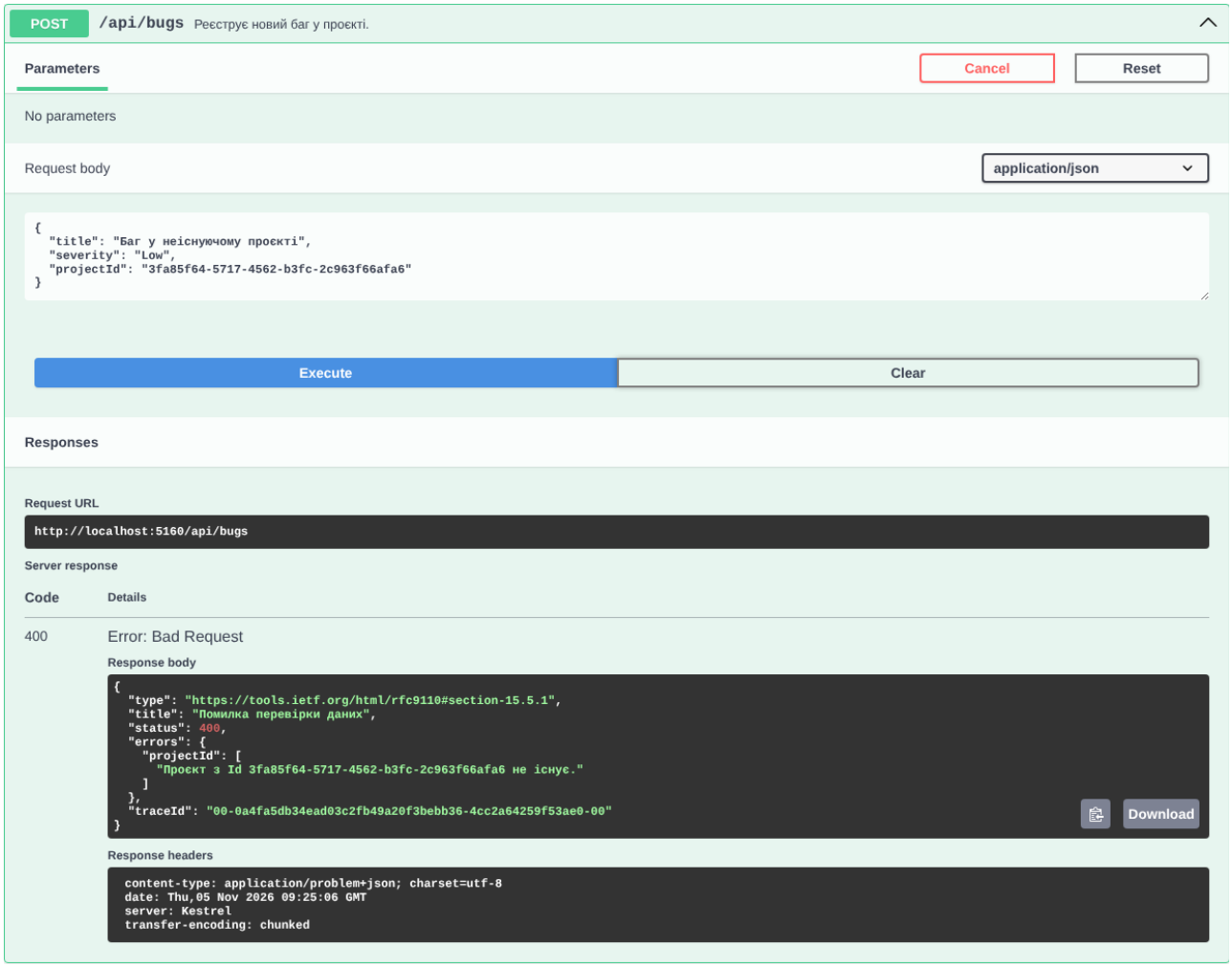


Рисунок 20 — POST /api/bugs: 400 Bad Request

Запит 19. PUT /api/bugs/{id} → 400 Bad Request

Спроба позначити баг виправленим, доки його нікому не призначено.

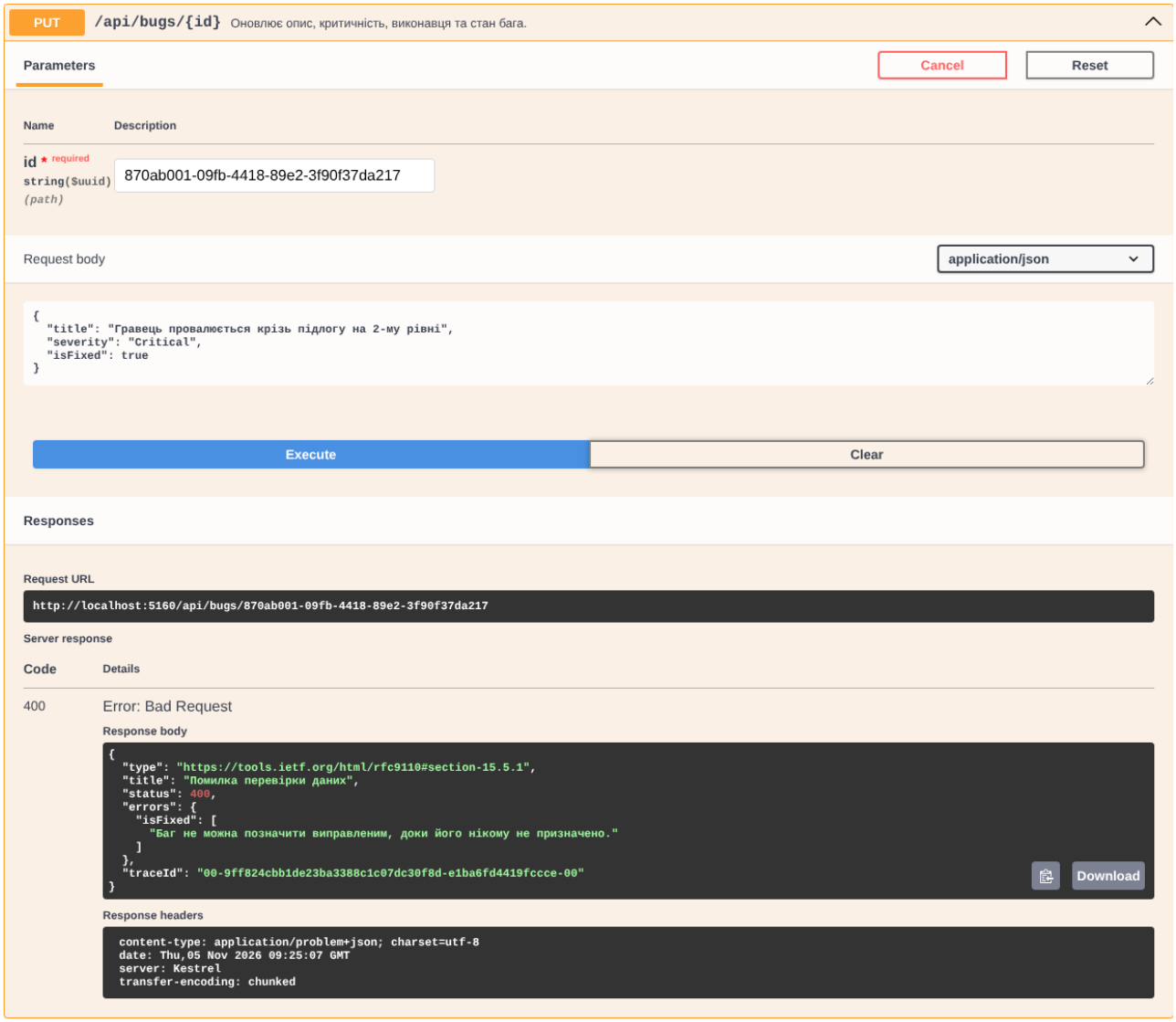


Рисунок 21 — PUT /api/bugs/{id}: 400 Bad Request

Запит 20. PUT /api/bugs/{id} → 200 OK

Призначення виконавця і позначення бага виправленим.

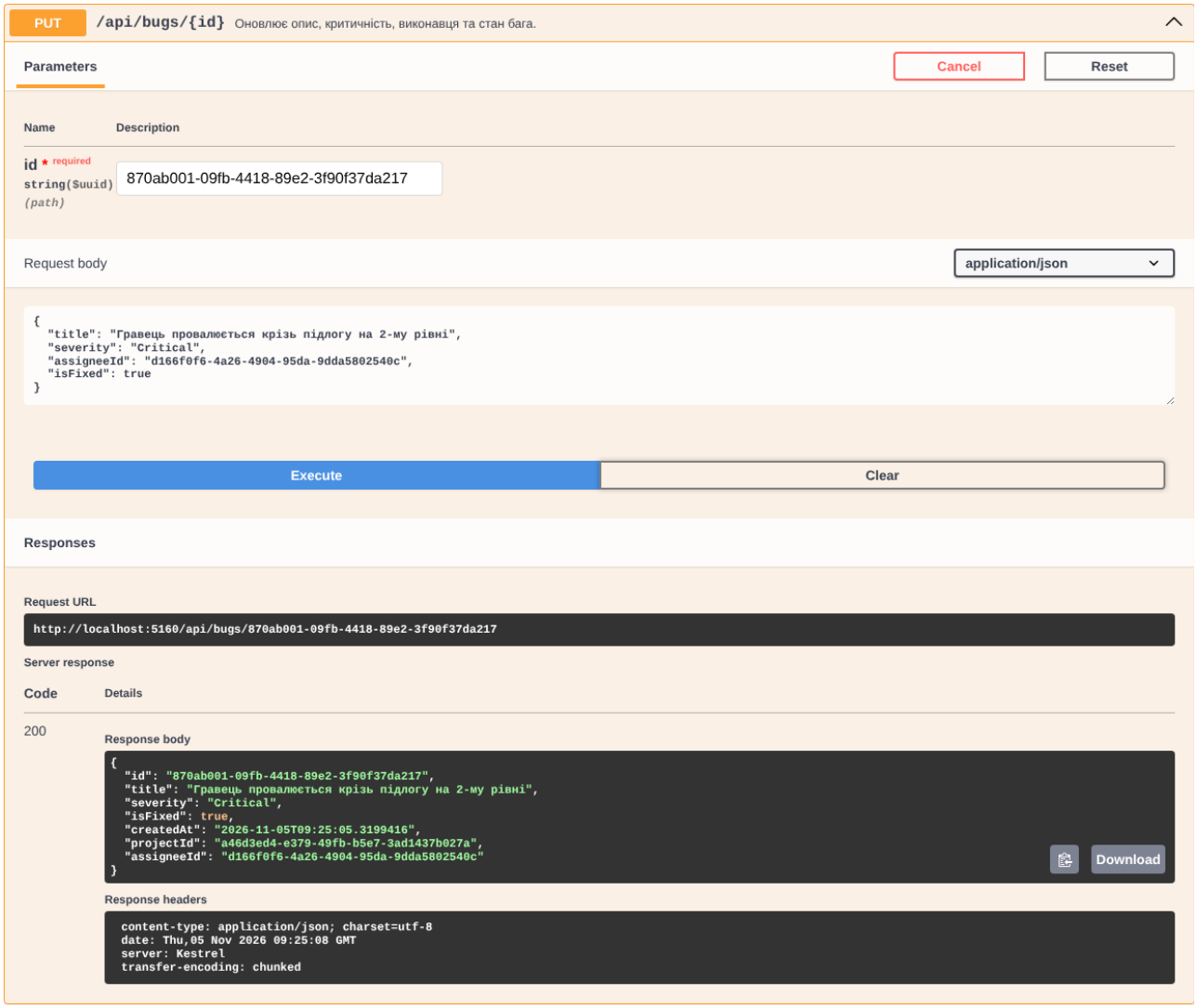


Рисунок 22 — PUT /api/bugs/{id}: 200 OK

Запит 21. GET /api/projects/{id}/bugs → 200 OK

Баги проєкту (зв'язок 1:N).

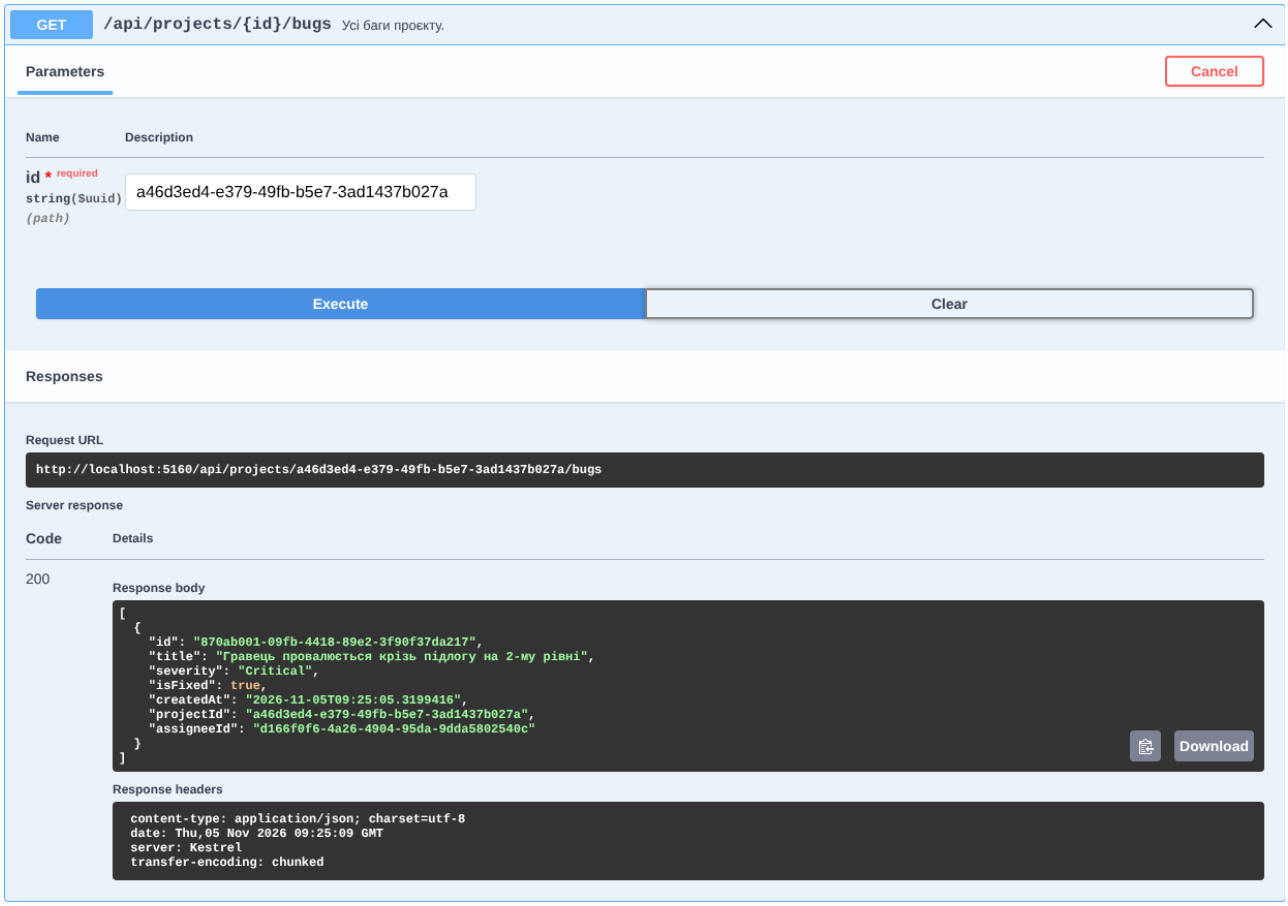


Рисунок 23 — GET /api/projects/{id}/bugs: 200 OK

Запит 22. PUT /api/programmers/{id} → 400 Bad Request

Оновлення програміста з некоректними даними.

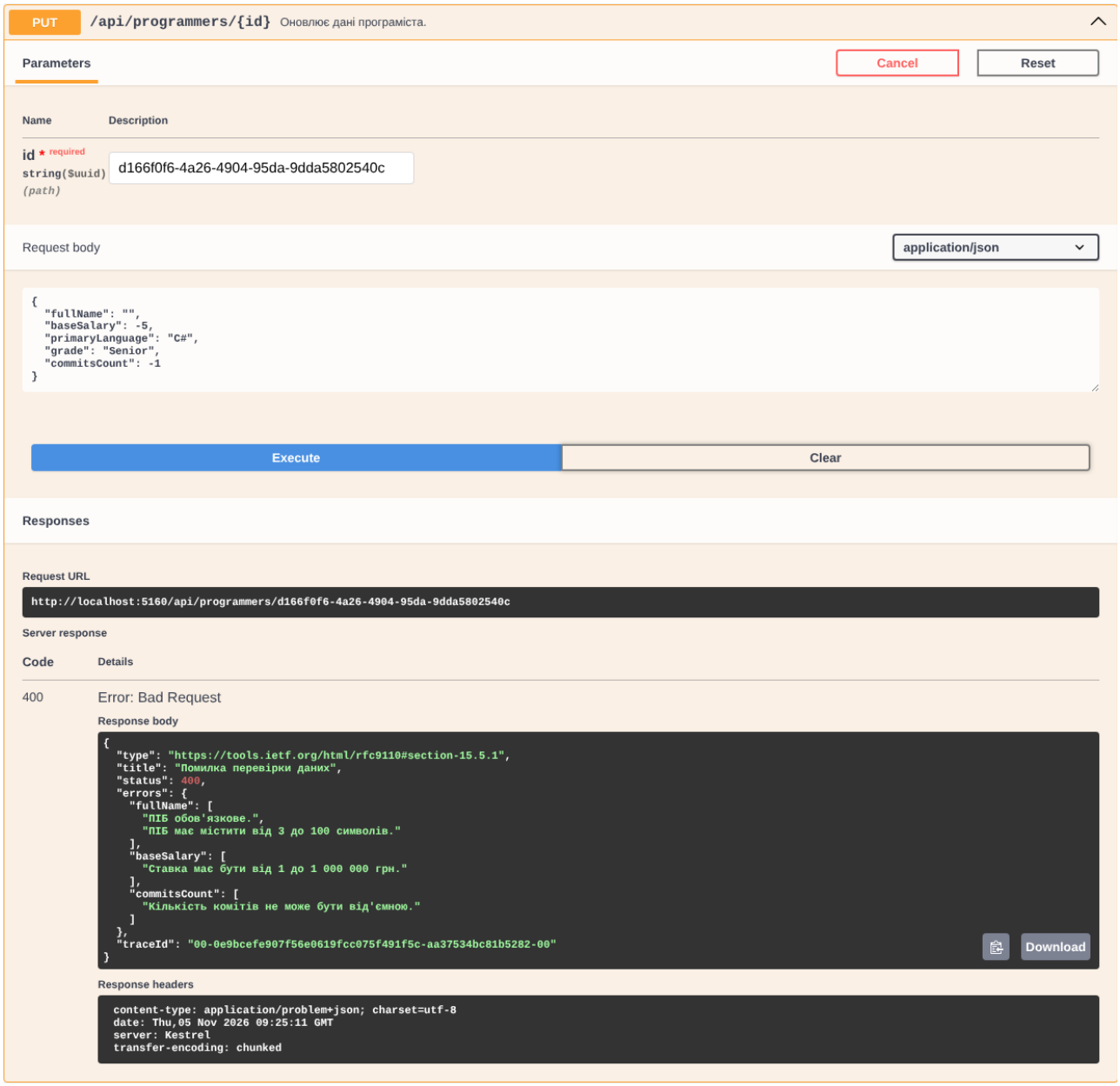


Рисунок 24 — PUT /api/programmers/{id}: 400 Bad Request

Запит 23. PUT /api/projects/{id} → 200 OK

Випуск гри: статус Released.

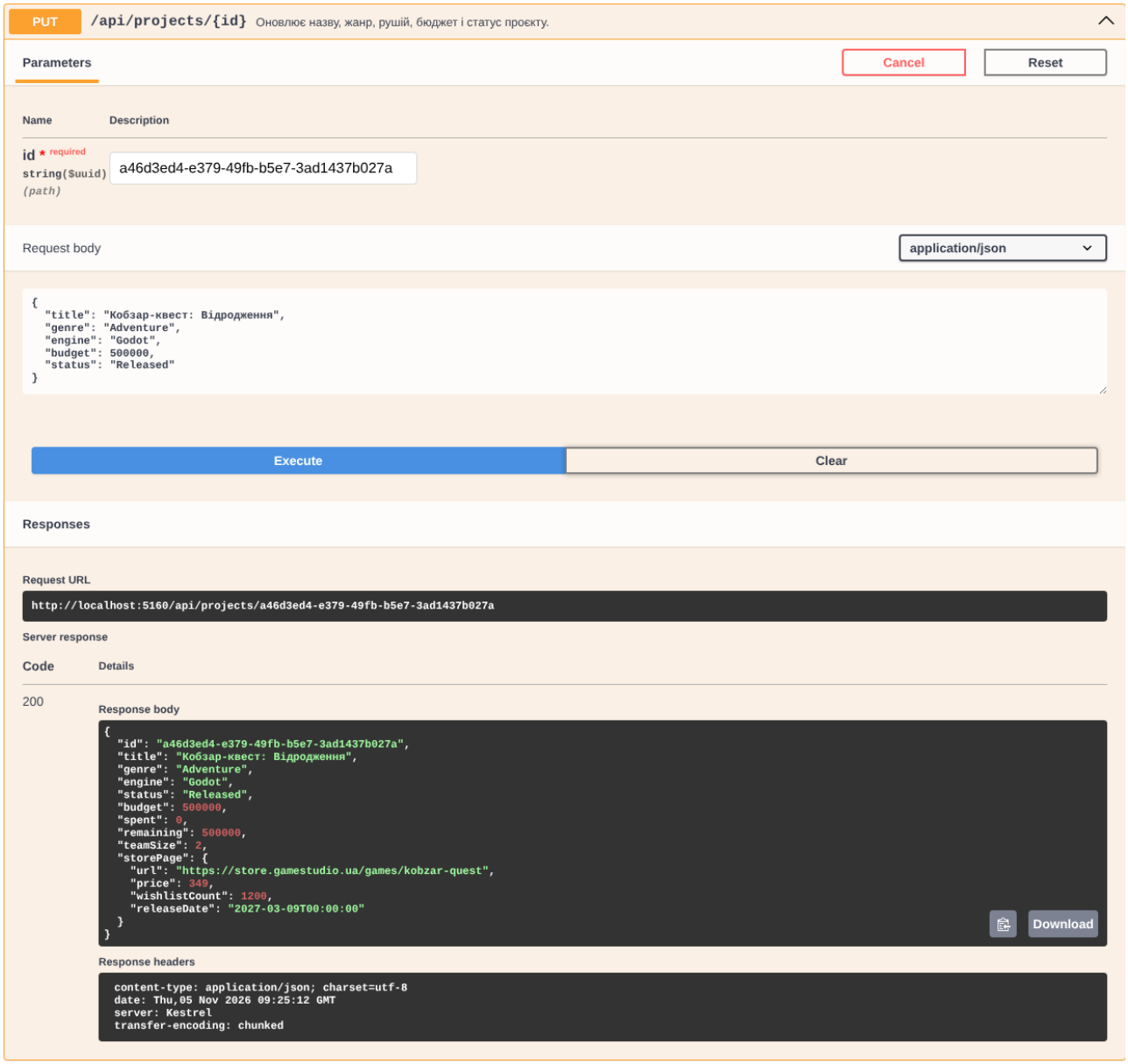


Рисунок 25 — PUT /api/projects/{id}: 200 OK

Запит 24. PUT /api/projects/{id} → 409 Conflict

Спроба змінити статус уже випущеного проєкту.

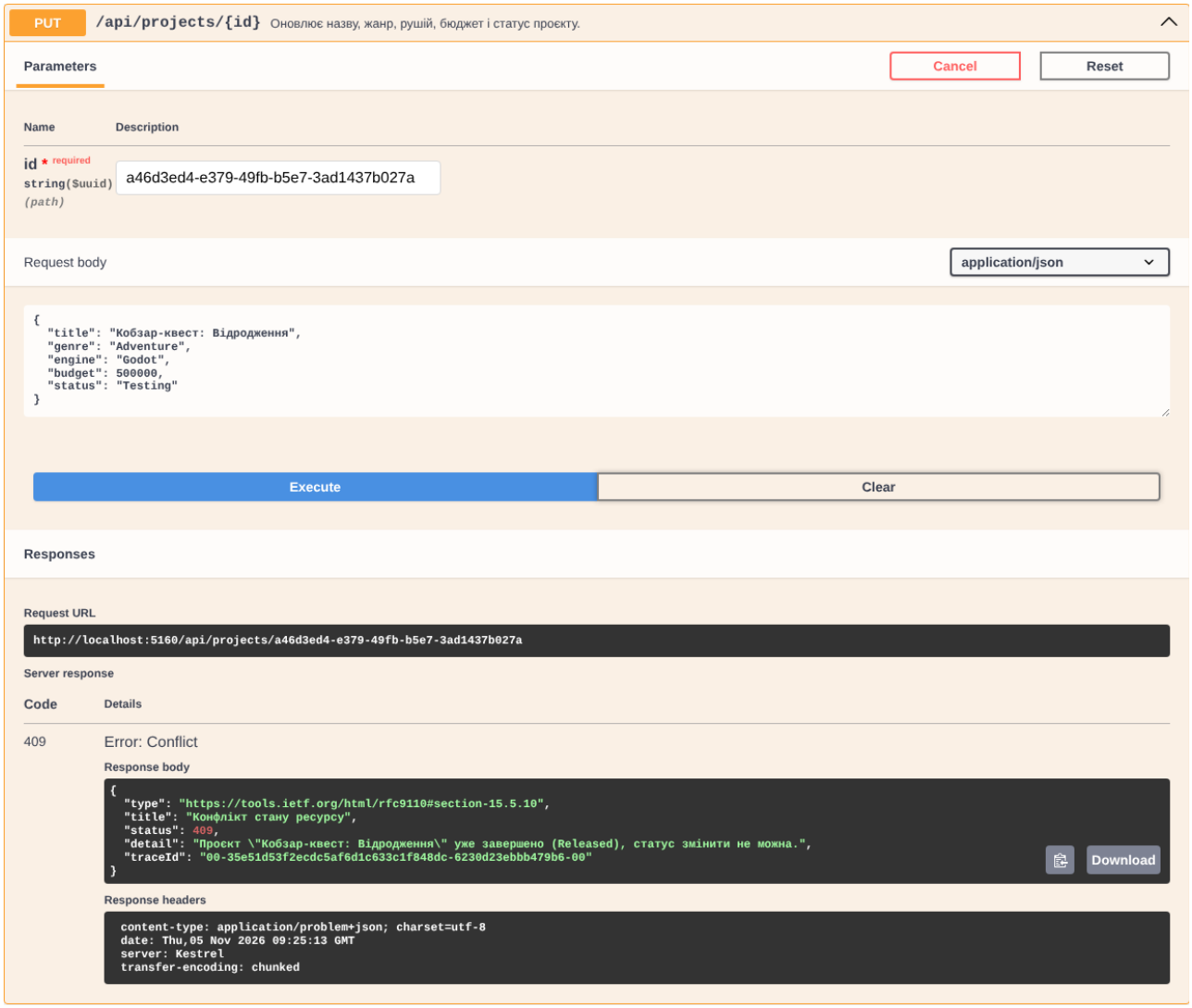


Рисунок 26 — PUT /api/projects/{id}: 409 Conflict

Запит 25. DELETE /api/projects/{id}/team/{employeeId} → 204 No Content

Вилучення художника з команди.

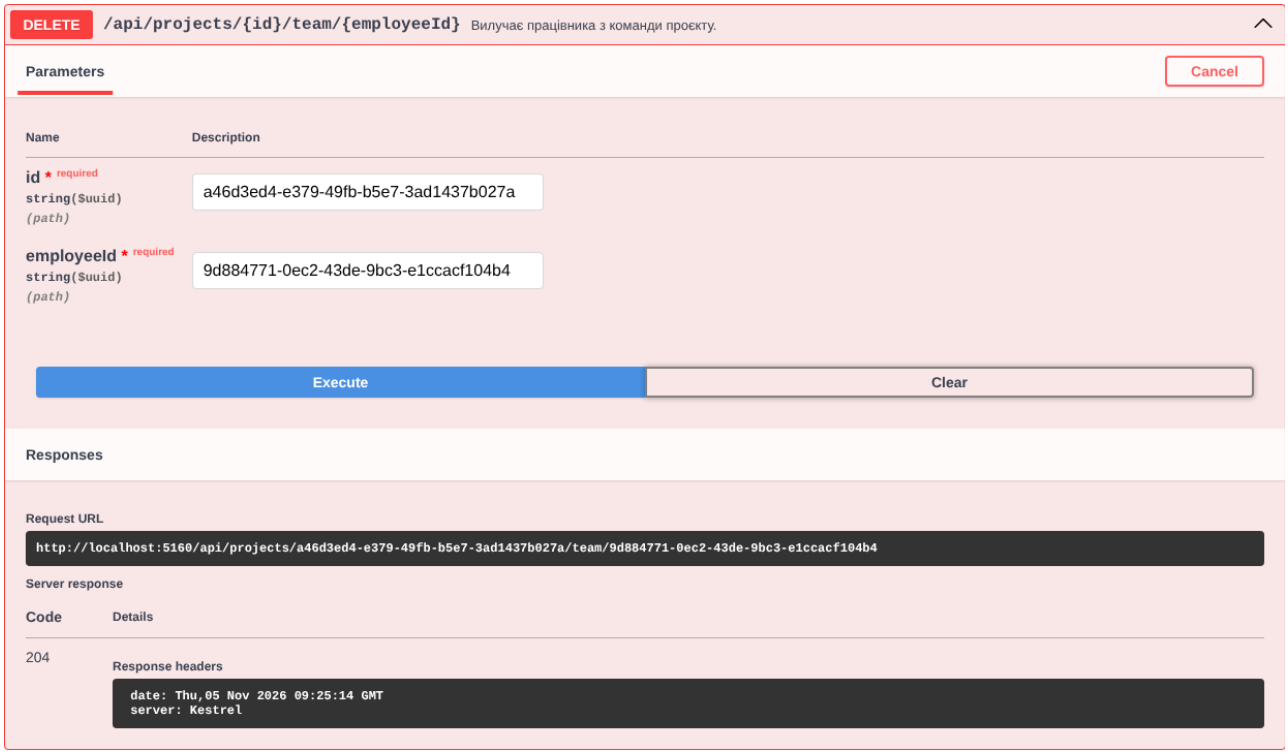


Рисунок 27 — DELETE /api/projects/{id}/team/{employeeId}: 204 No Content

Запит 26. DELETE /api/bugs/{id} → 204 No Content

Видалення бага.

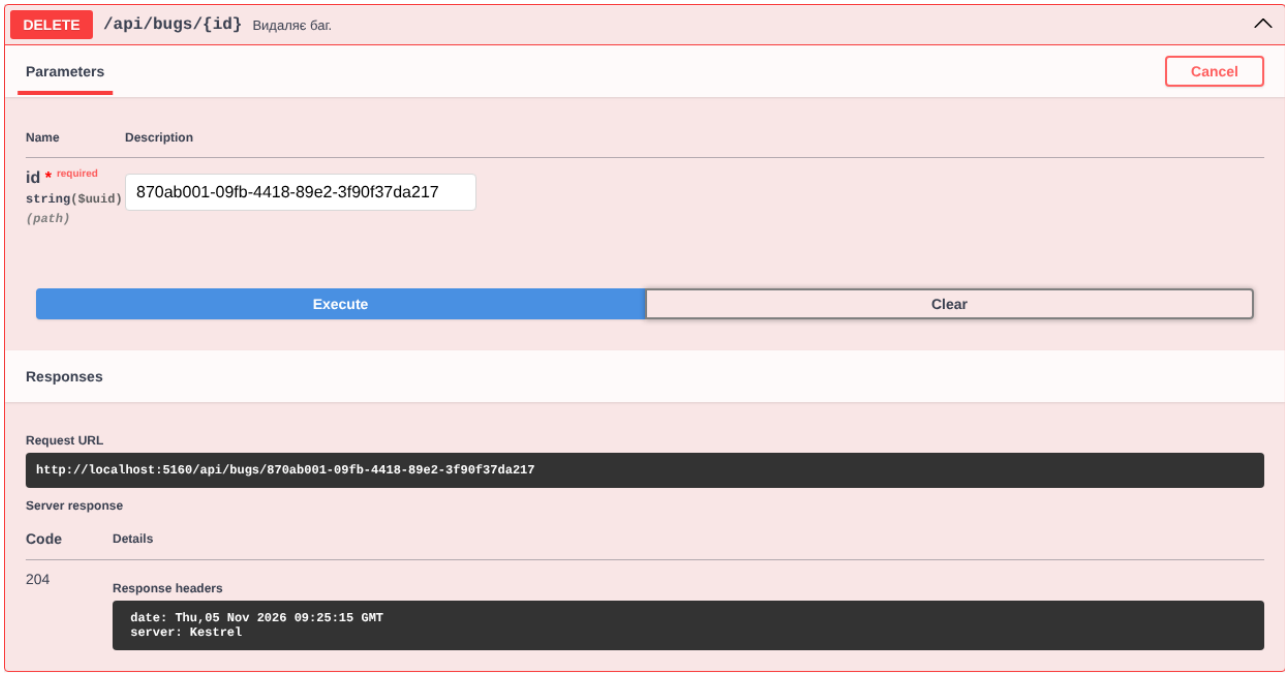


Рисунок 28 — DELETE /api/bugs/{id}: 204 No Content

Запит 27. DELETE /api/projects/{id} → 204 No Content

Видалення проєкту (сторінка в магазині та записи команди видаляються каскадно).

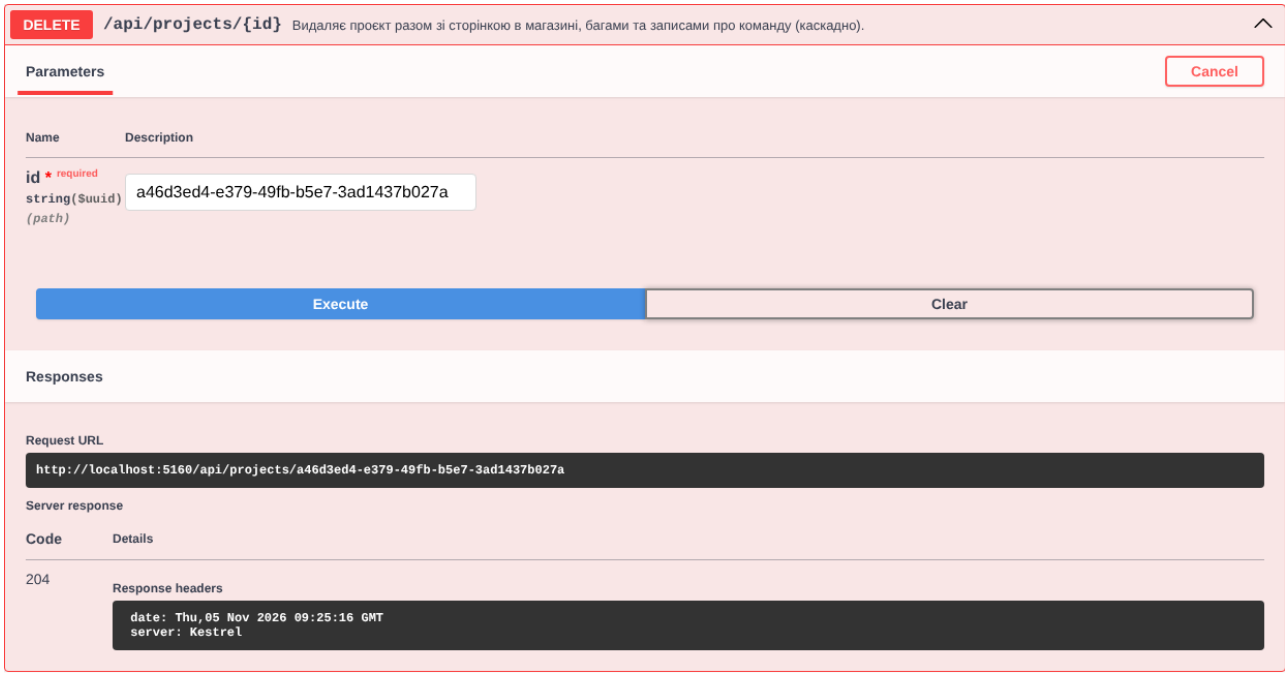


Рисунок 29 — DELETE /api/projects/{id}: 204 No Content

Запит 28. GET /api/projects/{id} → 404 Not Found

Читання видаленого проєкту.

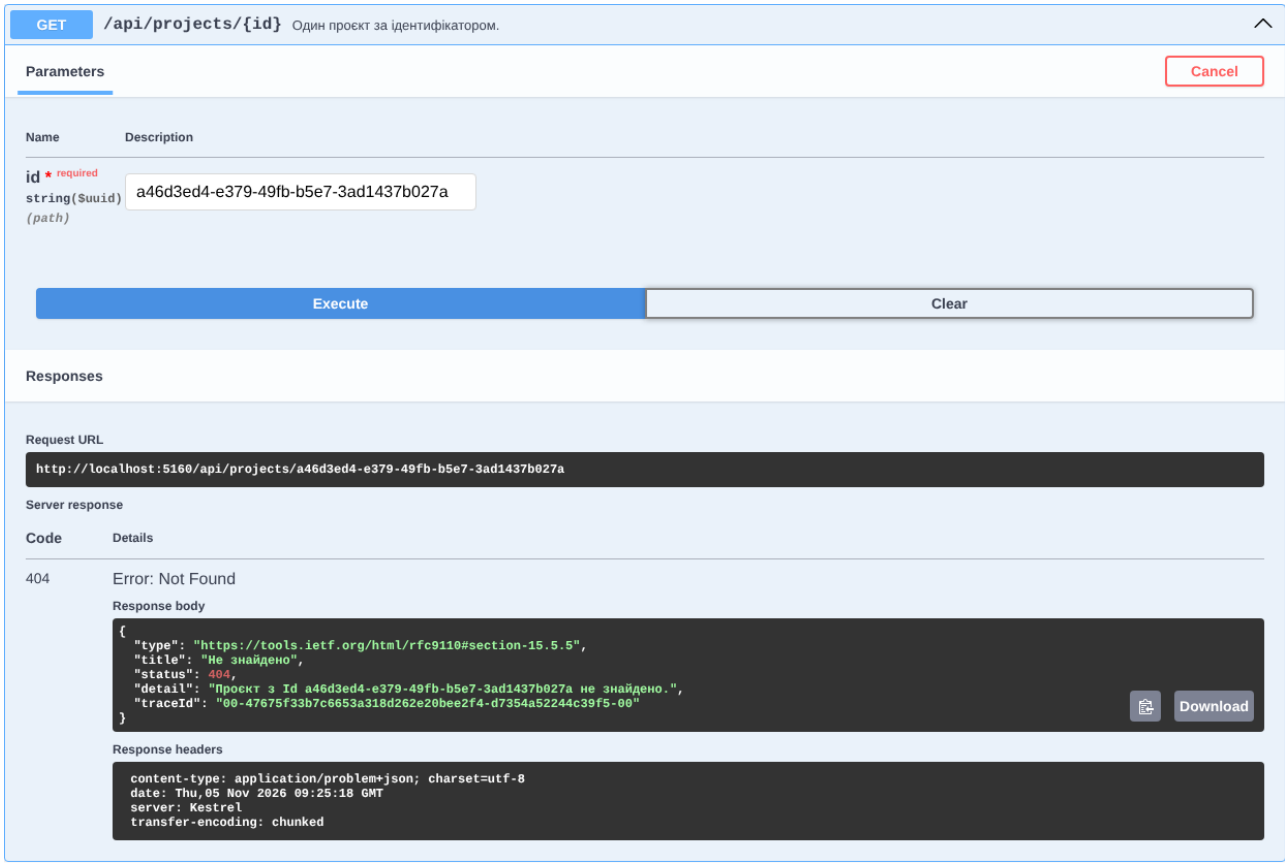


Рисунок 30 — GET /api/projects/{id}: 404 Not Found

Запит 29. DELETE /api/programmers/{id} → 204 No Content

Звільнення програміста.

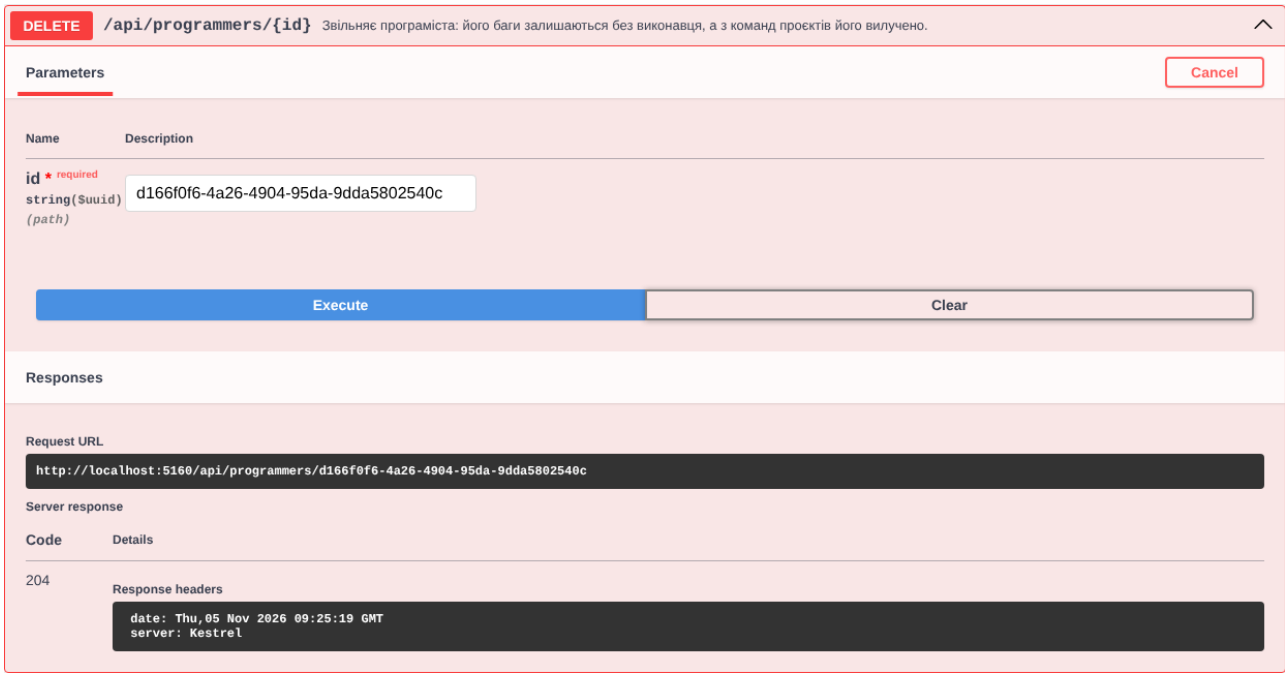


Рисунок 31 — DELETE /api/programmers/{id}: 204 No Content

Запит 30. DELETE /api/artists/{id} → 204 No Content

Звільнення художника.

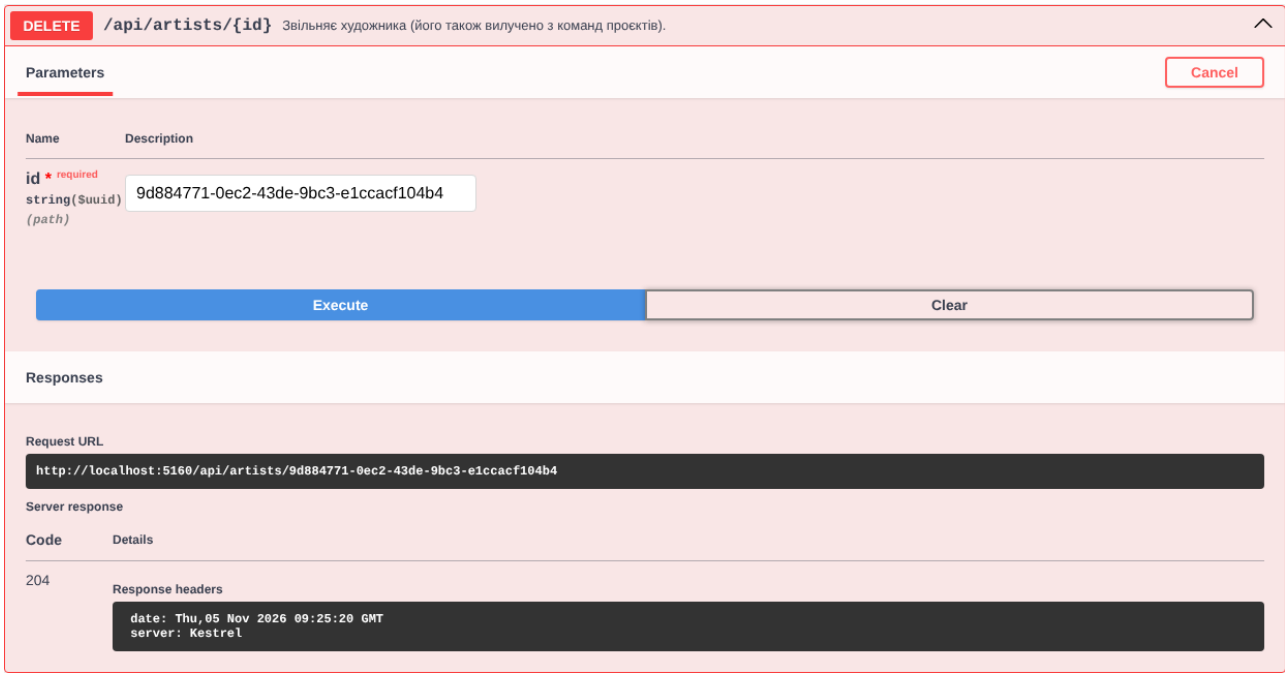


Рисунок 32 — DELETE /api/artists/{id}: 204 No Content

3. Додаткове завдання: вебзастосунок GameStudio.MVC

Проєкт створено за шаблоном ASP.NET Core Web App (Model-View-Controller): `dotnet new mvc -n GameStudio.MVC`. Він посилається на `GameStudio.Infrastructure`, а контекст БД з ЛР3 зареєстровано в `Program.cs` командою `builder.Services.AddDbContext<GameStudioContext>(options => options.UseSqlite(connectionString))`. Сторінки згенеровано вбудованим генератором ASP.NET (у Visual Studio: Add → New Scaffolded Item → MVC Controller with views, using Entity Framework) або з командного рядка:

```
dotnet tool install dotnet-aspnet-codegenerator --version 8.0.23
dotnet aspnet-codegenerator controller -name ProjectsController \
    -m GameStudio.Infrastructure.Models.GameProjectModel \
    -dc GameStudio.Infrastructure.GameStudioContext -dbProvider sqlite \
    --relativeFolderPath Controllers --useDefaultLayout --referenceScriptLibraries
# так само: ProgrammersController (ProgrammerModel), ArtistsController (ArtistModel),
#           BugsController (BugReportModel)
```

Для кожної сутності генератор створив контролер з діями Index, Details, Create, Edit, Delete та п'ять Razor-представлень. Після генерації код доопрацьовано:

- списки вибору для переліків (`Html.GetEnumSelectList<ProjectStatus>()` тощо): генератор створює їх порожніми;
- `PublicId` не вводиться у формі, а генерується в контролері (`Guid.NewGuid()`); у формі бага проєкт обирається за назвою, а не за рушієм;
- підписи полів і повідомлення перевірки українською — через власний `IDisplayMetadataProvider`, без змін у моделях рівня доступу до даних;
- пагінація списків (`Paging.ToPageAsync`, частковий вигляд `_Pager`), головна сторінка зі статистикою, меню розділів;
- на сторінці проєкту показано пов'язані дані з ЛР3: сторінку в магазині, команду та баги.

Запуск: `dotnet run --project GameStudio.MVC` (профіль `http`, адреса `http://localhost:5110`). Скріншоти зроблено на копії БД.

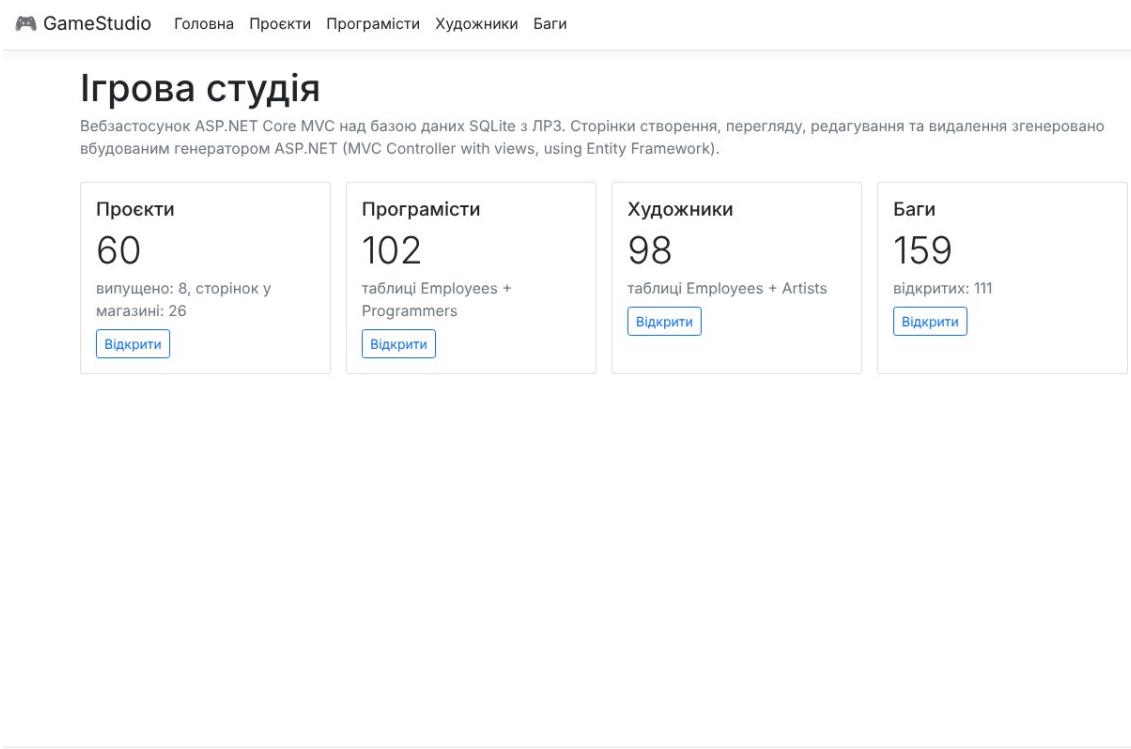


Рисунок 33 — Головна сторінка GameStudio.MVC: кількість записів у таблицях БД

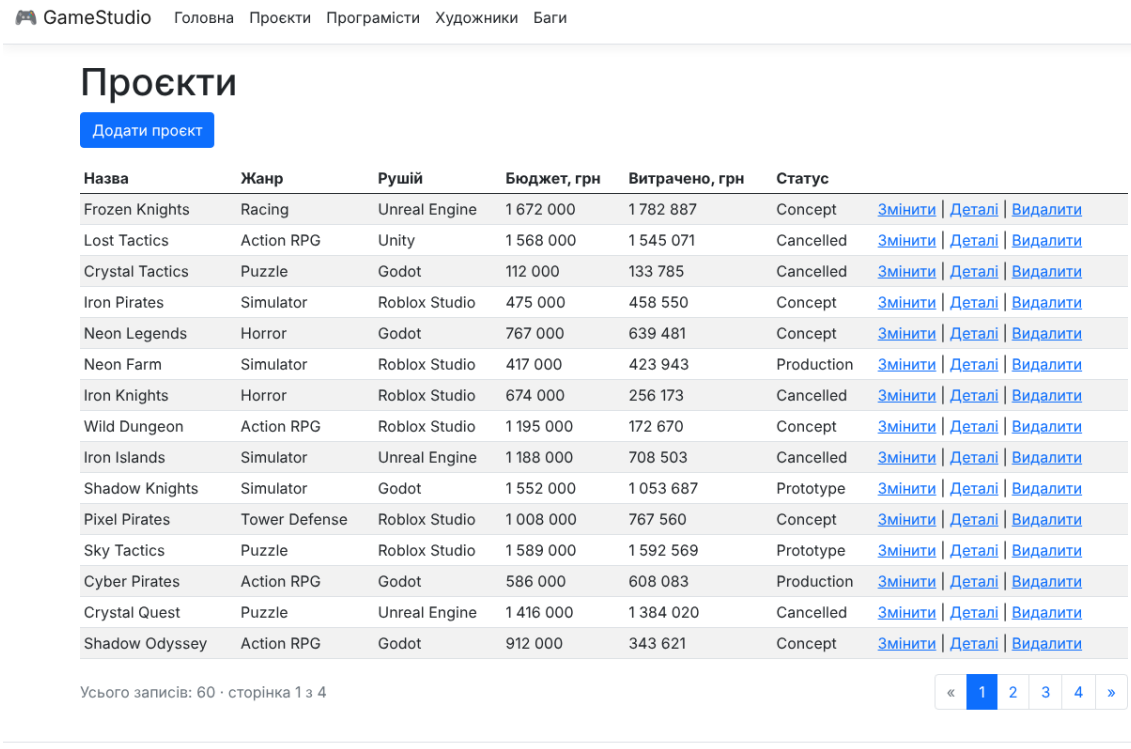


Рисунок 34 — Список проєктів (Projects/Index) з пагінацією по 15 записів

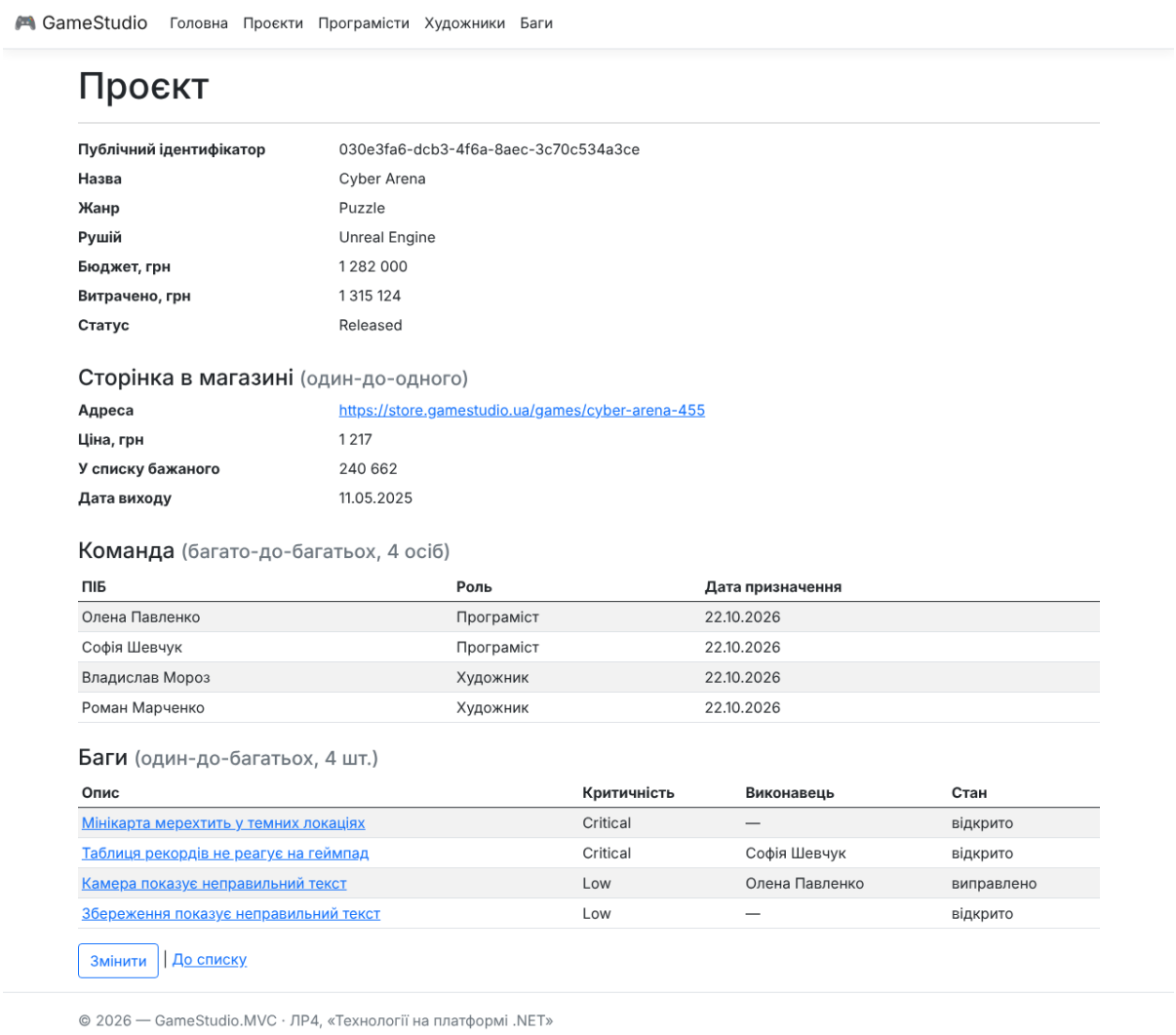


Рисунок 35 — Сторінка проєкту (Projects/Details): дані проєкту, сторінка в магазині (1:1), команда (N:M), баги (1:N)

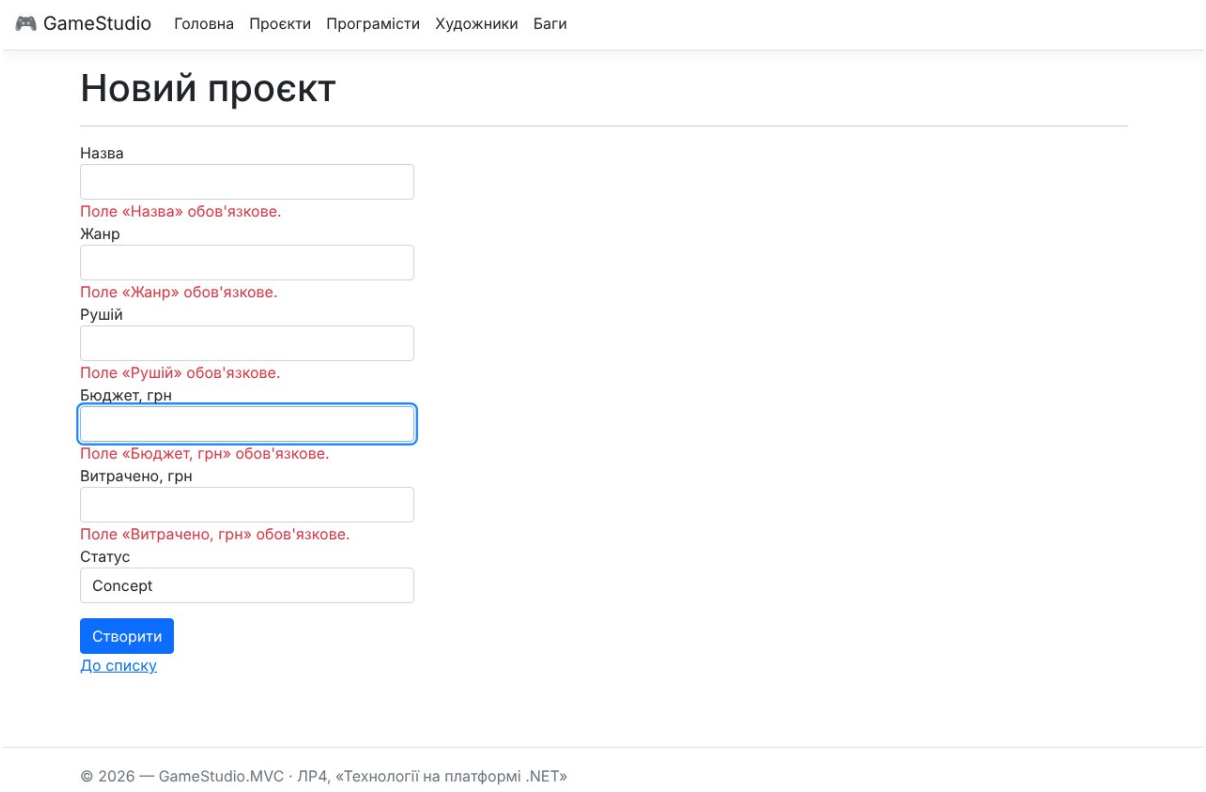


Рисунок 36 — Створення проєкту: перевірка обов'язкових полів у браузері

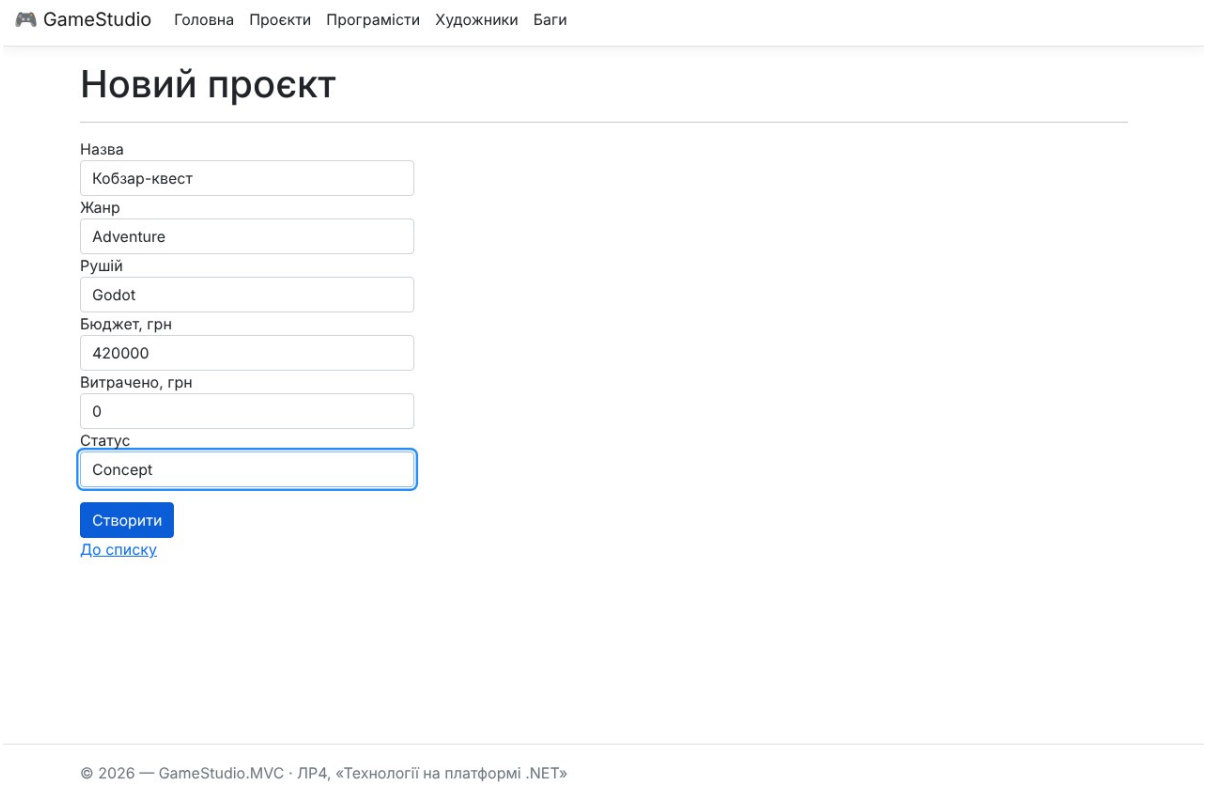


Рисунок 37 — Створення проєкту: заповнена форма

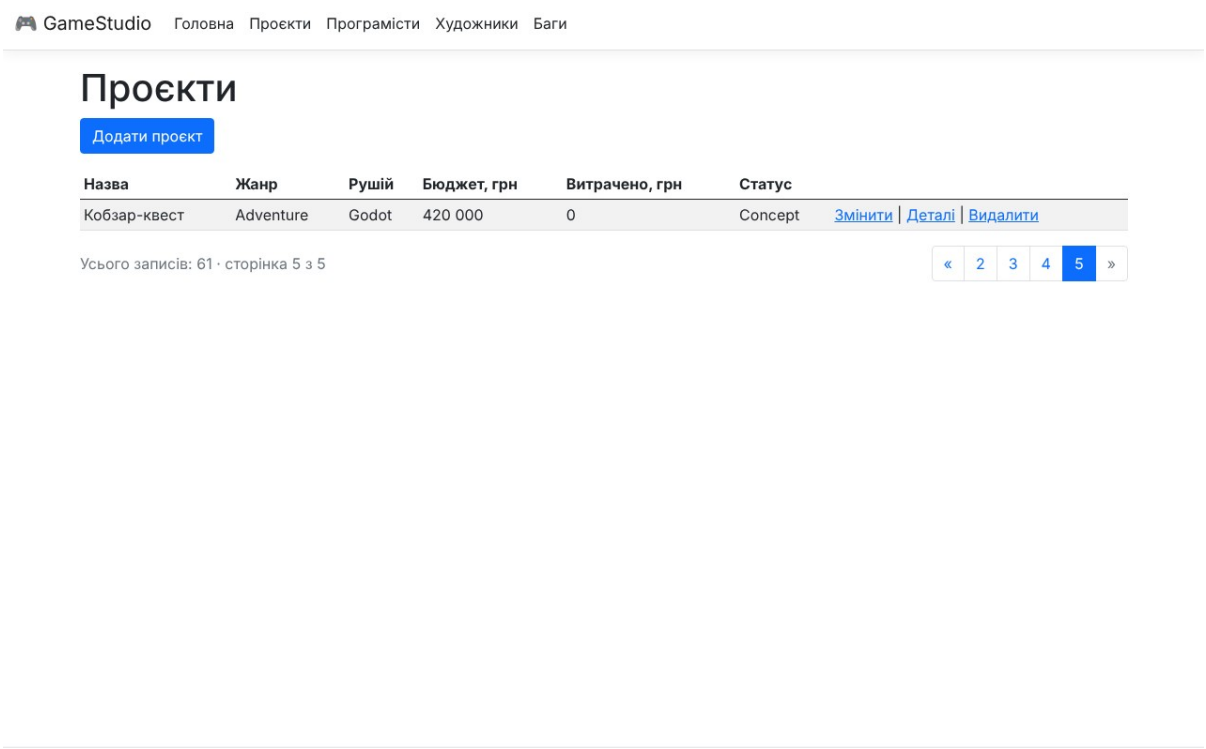


Рисунок 38 — Останню сторінку списку доповнено новим проєктом

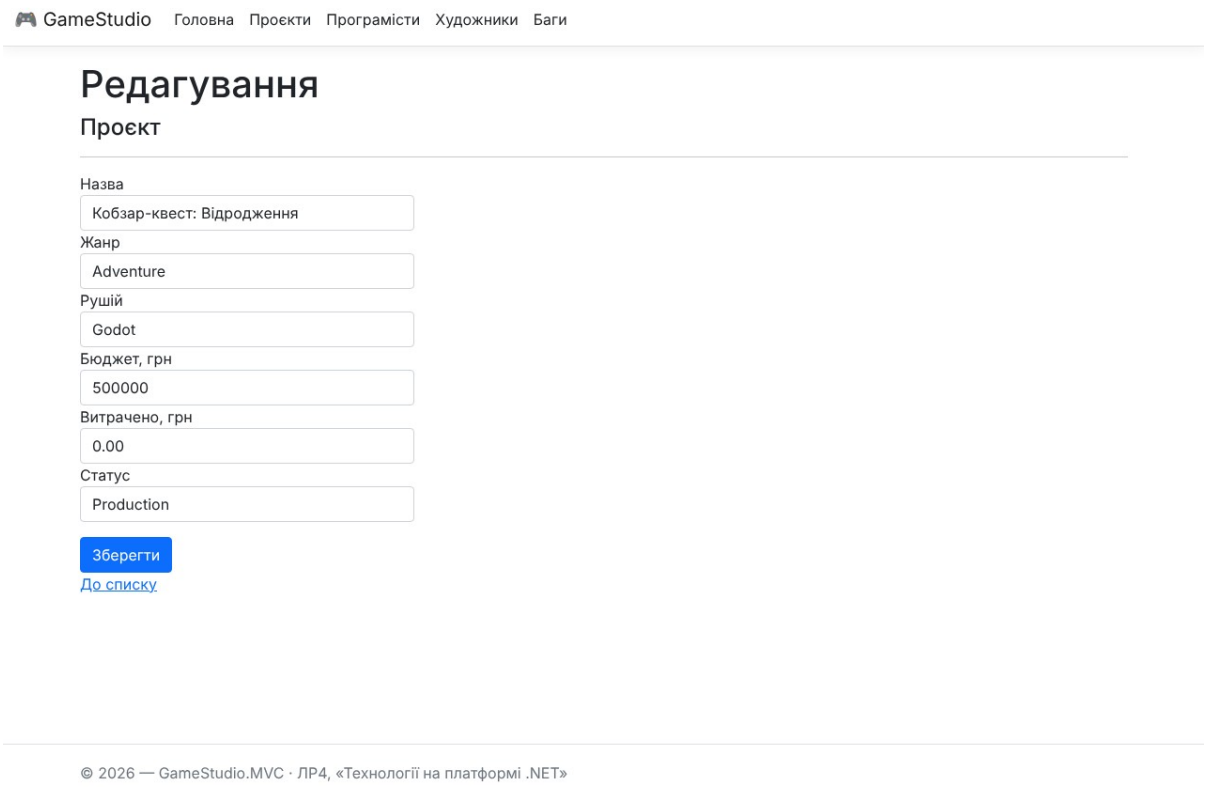


Рисунок 39 — Редагування проєкту (Projects/Edit): нова назва, бюджет і статус

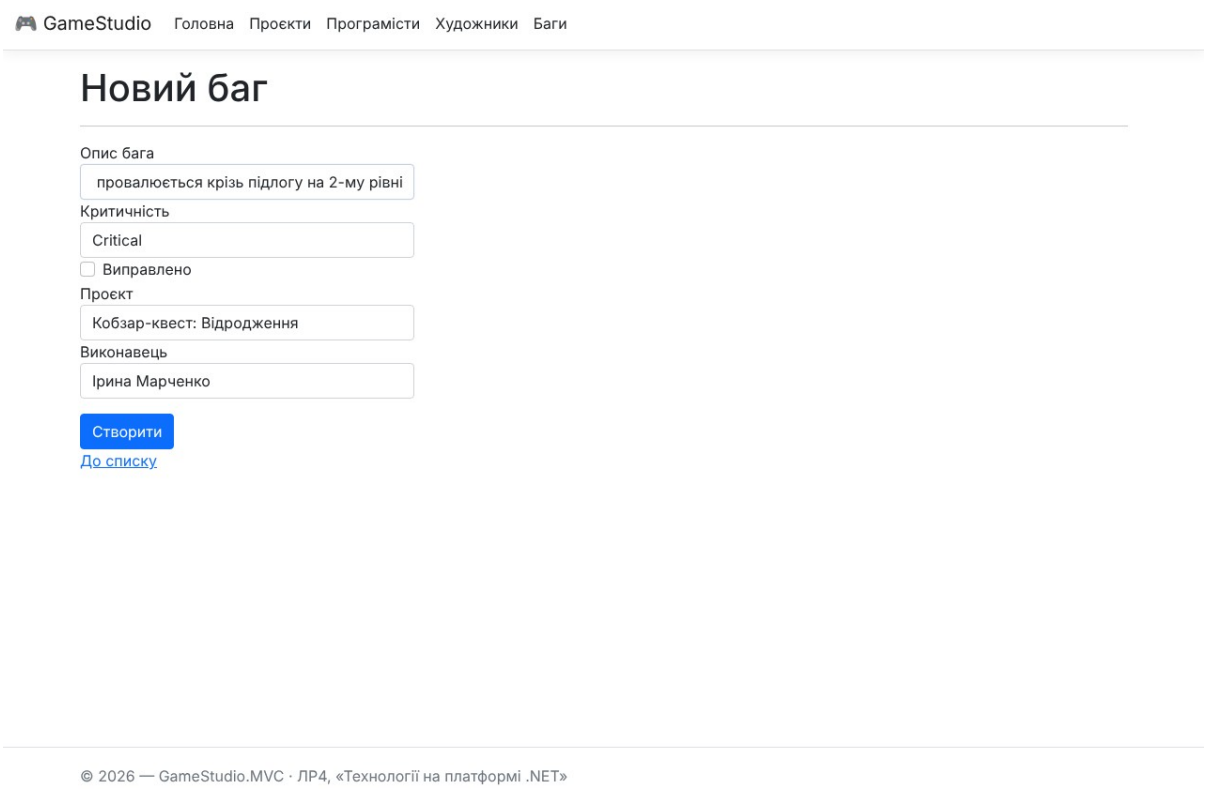


Рисунок 40 — Створення бага: списки вибору проєкту (за назвою) і виконавця

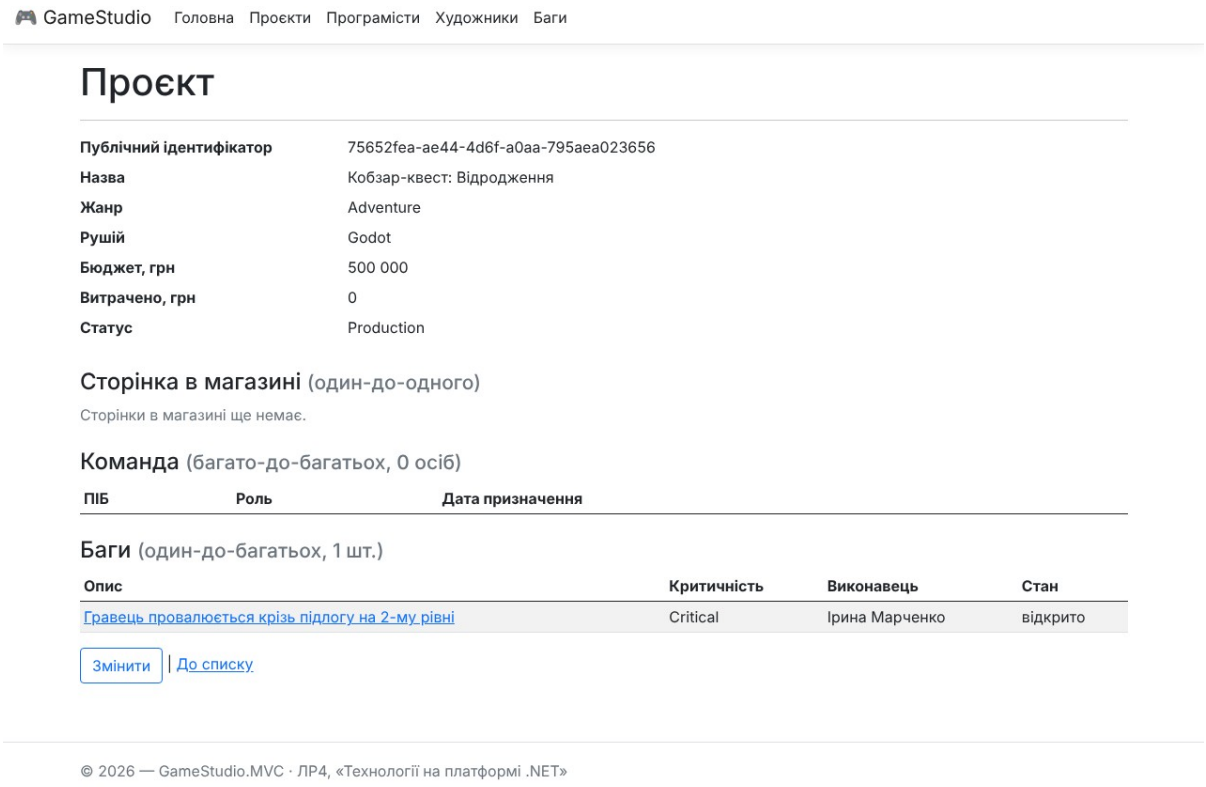


Рисунок 41 — Сторінка нового проєкту після змін: доданий баг відображається у зв'язку 1:N

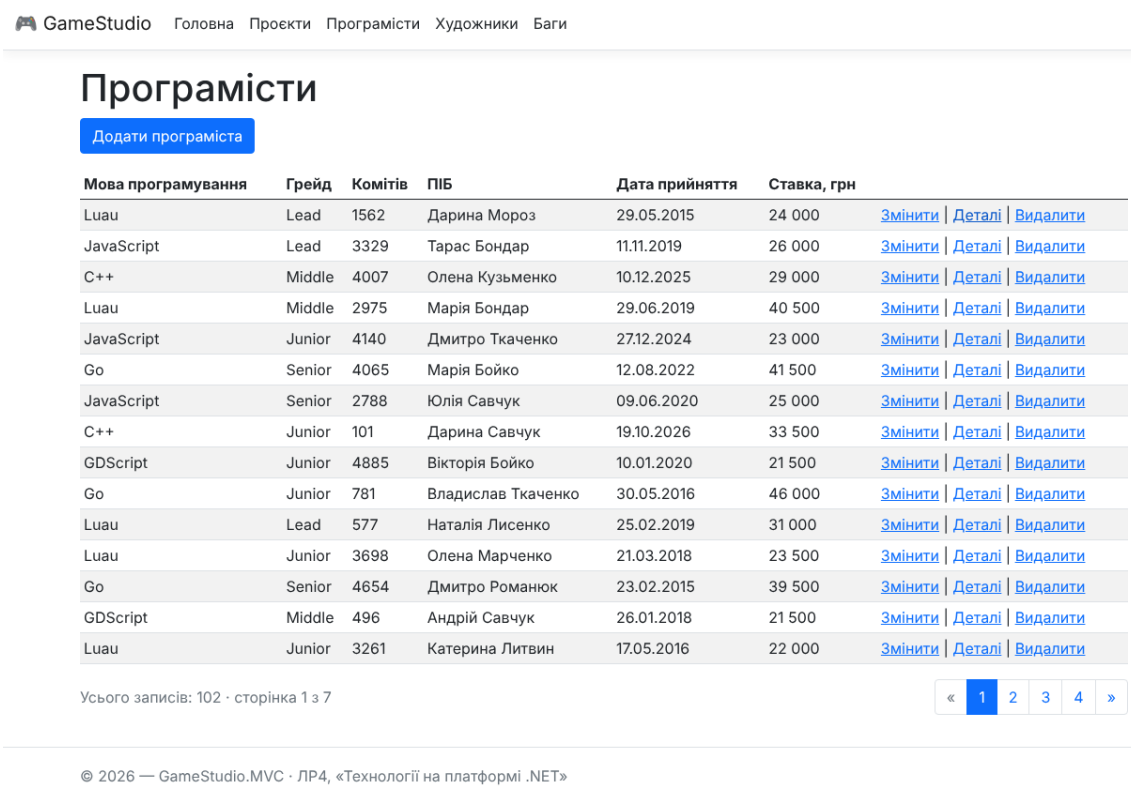


Рисунок 42 — Список програмістів (таблиці Employees + Programmers, TPT)

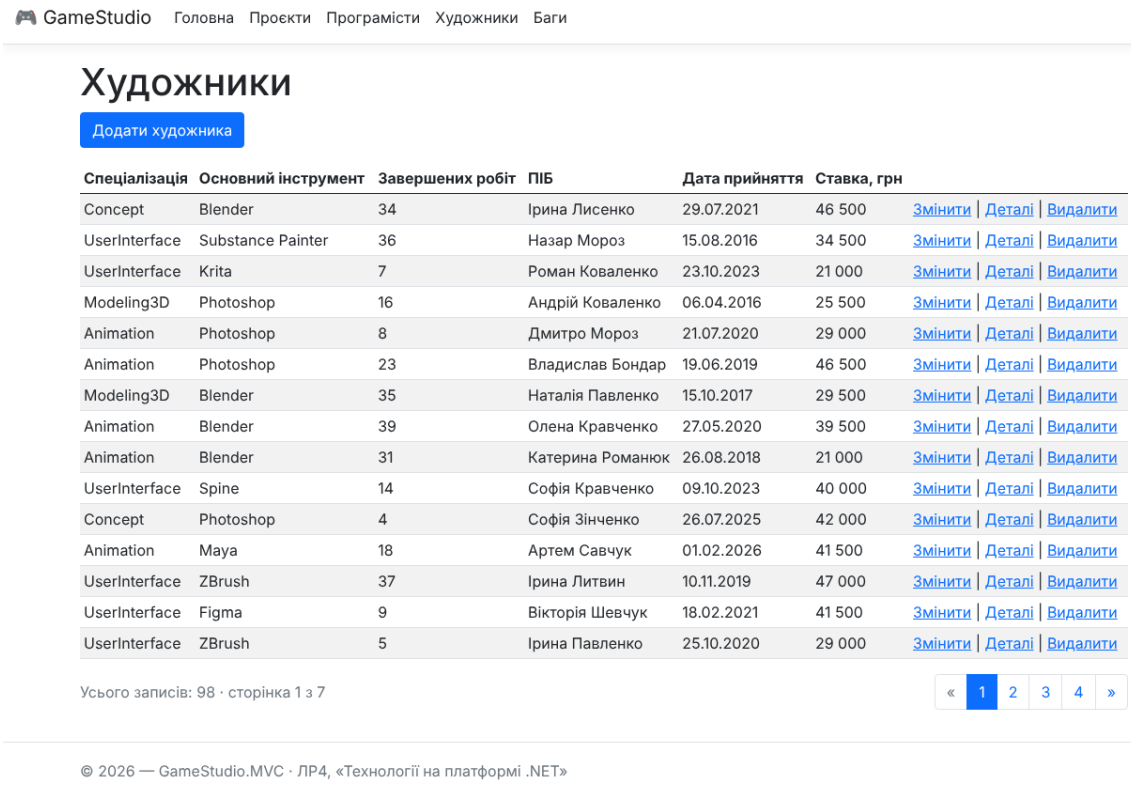


Рисунок 43 — Список художників (таблиці Employees + Artists, TPT)

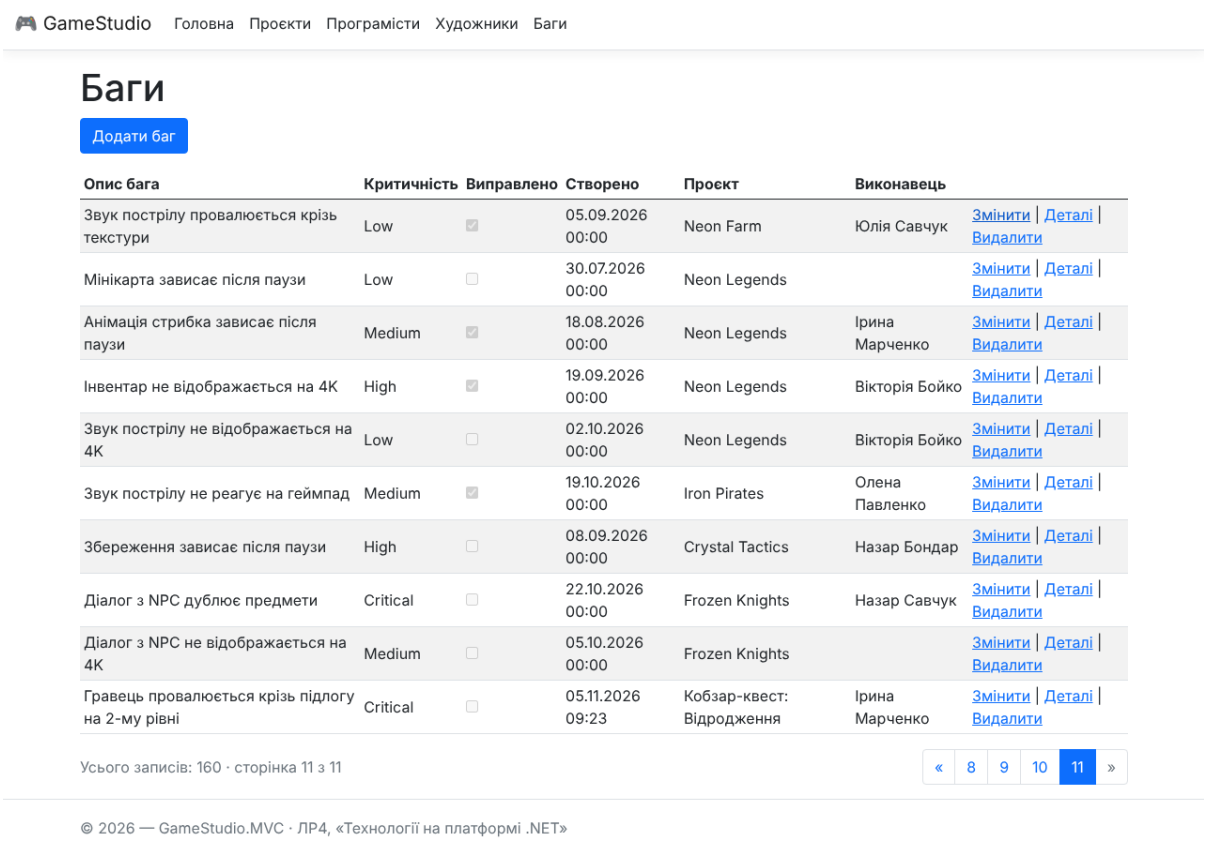


Рисунок 44 — Список багів, остання сторінка: новий баг з назвою проєкту і виконавцем

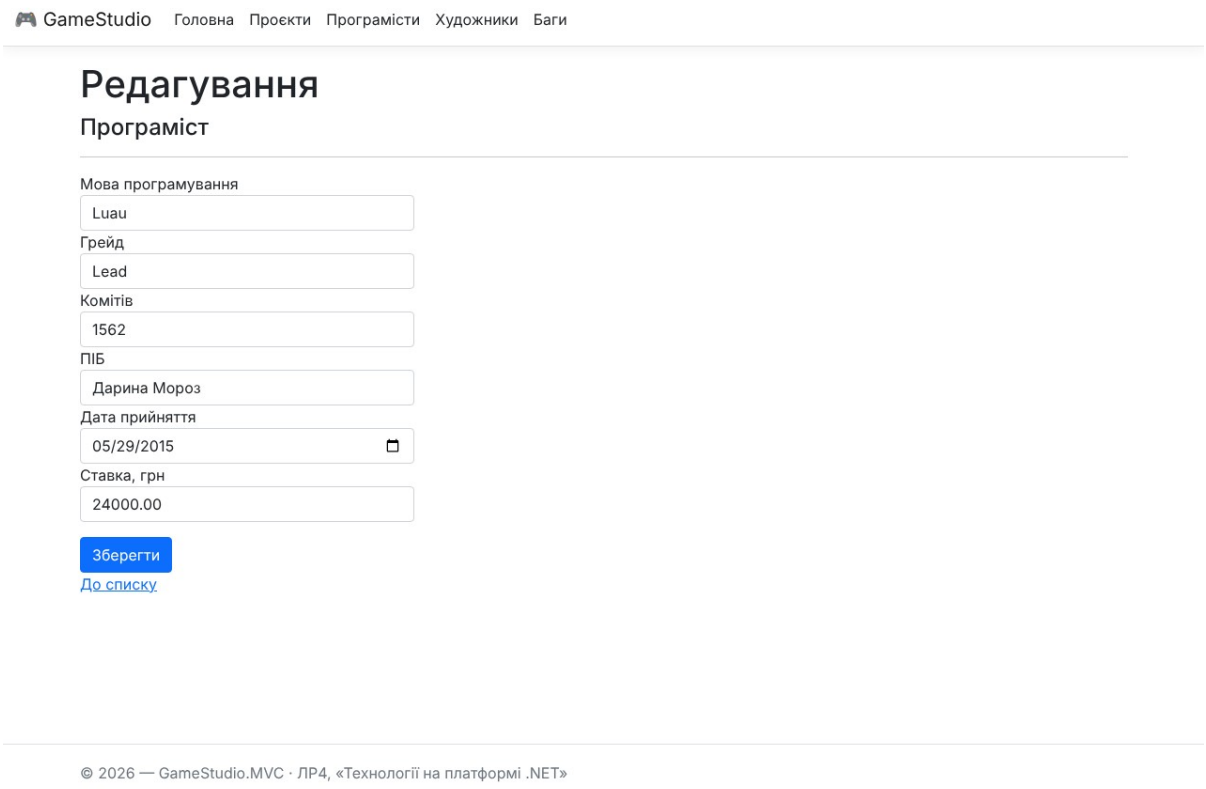


Рисунок 45 — Редагування програміста: грейд обирається зі списку, дата — у полі дати

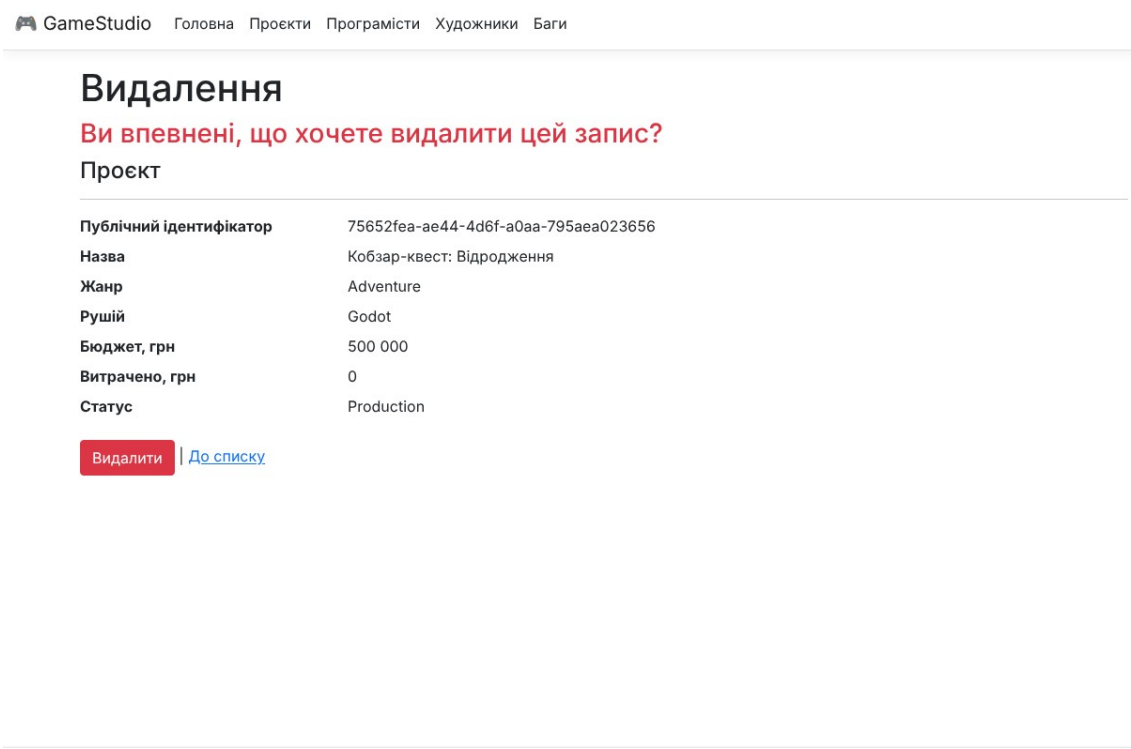


Рисунок 46 — Підтвердження видалення проєкту (Projects/Delete)

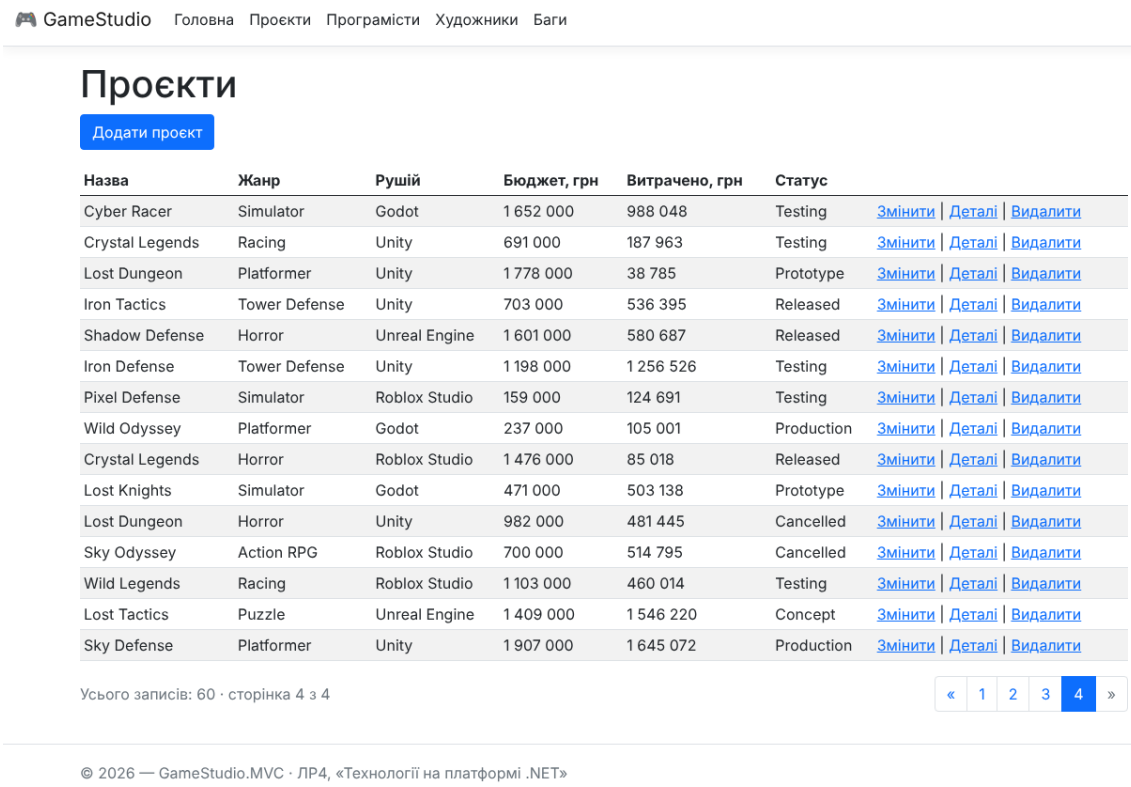


Рисунок 47 — Список після видалення: проєкт (і каскадно його баг) видалено

4. Результати тестів

Команда `dotnet test`: пройдено 61 з 61 тестів. Інтеграційні тести REST API (`GameStudio.Tests/RestApiTests.cs`):

```
✔ RestApiTests.Api_UsesTemporaryDatabase
✔ RestApiTests.CreateProject_Returns201WithLocation_AndCanBeReadBack
✔ RestApiTests.CreateProject_WithInvalidData_Returns400WithFieldErrors
✔ RestApiTests.UnknownIds_Return404ForEveryResource
✔ RestApiTests.Pagination_ReturnsRequestedPage_AndRejectsInvalidParameters
✔ RestApiTests.UpdateProject_Returns200_AndFinishedProjectStatusChangeReturns409
✔ RestApiTests.DeleteProject_Returns204_AndCascadesToBugs
✔ RestApiTests.StorePage_PutCreatesThenReplaces_DeleteRemoves
✔ RestApiTests.Team_AddAndRemoveMembers
✔ RestApiTests.Bugs_ValidateReferences_AndCanBeFixedOnlyWithAssignee
✔ RestApiTests.DeleteProgrammer_Returns204_AndLeavesBugsWithoutAssignee
✔ RestApiTests.Programmer_UpdateRecalculatesComputedFields

Total tests: 61
     Passed: 61
```

5. Відповіді на контрольні запитання

1. Які ключові особливості шаблону ASP.NET Core Web API та чим він відрізняється від Blazor Web App або ASP.NET MVC?

Шаблон **ASP.NET Core Web API** (`dotnet new webapi --use-controllers`) створює HTTP-сервіс, який повертає дані (JSON), а не HTML. Його ключові особливості:

- контролери успадковують `ControllerBase` (без підтримки представлень) і позначені атрибутом `[ApiController]`;
- `[ApiController]` вимагає маршрутизації атрибутами (`[Route("api/projects")]`, `[HttpGet("{id:guid}")]`), сам визначає джерело параметрів (тіло, шлях, рядок запиту) і автоматично повертає 400 з описом помилок, якщо модель не пройшла перевірку;
- помилки повертаються у форматі ProblemDetails, результати дій — через `Ok()`, `CreatedAtAction()`, `NoContent()`, `NotFound()`;
- до шаблону .NET 8 входить Swagger (Swashbuckle): опис API у форматі OpenAPI та сторінка для виконання запитів;
- клієнтом може бути будь-яка програма: SPA, мобільний застосунок, інший сервіс.

	Web API	**ASP.NET MVC**	**Blazor Web App**
Результат	дані (JSON) + коди HTTP	HTML-сторінки, які генерує сервер	інтерфейс з компонентів Razor (.razor)
Основа	`ControllerBase`, `[ApiController]`	`Controller`, представлення .cshtml, `View()`	компоненти C# з розміткою
Маршрутизація	атрибути на контролерах	шаблон `{controller}/{action}/{id?}`	`@page` у компонентах
Інтерактивність	реалізує клієнт	перезавантаження сторінки, форми, JavaScript	C# на сервері (SignalR) або в браузері (WebAssembly)
Інструменти	Swagger, Postman	браузер	браузер

2. Поясніть, як реалізується трирівнева архітектура у вашому проєкті GameStudio.REST та які переваги вона має.

Рівні відповідають окремим проєктам (рисунок і таблиця в розділі 1.1):

- **рівень представлення** — `GameStudio.REST`: контролери приймають HTTP-запити, перевіряють моделі API, перетворюють їх на доменні об'єкти й формують відповіді з правильними кодами;
- **рівень бізнес-логіки** — `GameStudio.Common`: доменні моделі з правилами (не можна змінити статус випущеного проєкту, не можна виправити баг без виконавця, розрахунок зарплати) і асинхронний CRUD-сервіс з ЛР3;

- **рівень доступу до даних** — `GameStudio.Infrastructure`: `GameStudioContext`, моделі та конфігурації БД, міграції, `Repository<T>`, перетворювачі доменна модель ↔ модель БД.

Залежності спрямовані лише вниз. Контролери працюють з інтерфейсом `ICrudServiceAsync<T>`, а про конкретні класи рівня доступу до даних знає тільки `Program.cs`, де реєструються залежності. **Переваги:** розділення відповідальності; можливість замінити СУБД чи репозиторій (наприклад, на MongoDB з ЛР3) без змін у контролерах; простіше тестування; повторне використання (ті самі Common та Infrastructure використовують консольний застосунок, REST API і MVC); клієнт не бачить внутрішньої структури БД.

3. Навіщо створювати окремі моделі у папці Models та в чому різниця між цими моделями й сутностями, що зберігаються в базі даних?

Моделі API (DTO) описують **контракт** API, тобто що клієнт надсилає й отримує. Моделі БД описують **структуру таблиць**. Причини їх розділити:

- у кожному методі — лише потрібні властивості: `ProjectCreateModel` не має `Id`, `Status` і `Spent` (їх задає сервер), `BugUpdateModel` не має `ProjectId`, бо проєкт бага не змінюється;
- захист від надмірного прив'язування (overposting): клієнт не може змінити поля, яких немає в моделі запиту, наприклад первинний ключ чи суму витрат;
- приховування деталей БД: цілочисельний `Id`, зовнішні ключі, проміжна сутність `ProjectAssignmentModel`, навігаційні властивості (до того ж вони утворюють цикли, що заважають серіалізації в JSON);
- у відповідях можуть бути обчислені поля (`Remaining`, `TeamSize`, `MonthlySalary`), яких немає в таблицях;
- схему БД можна змінювати (новою міграцією), не змінюючи контракт API для клієнтів.

Обмеження для моделей БД задано через Fluent API (`HasMaxLength`, `IsRequired`, зв'язки), а для моделей API — атрибутами перевірки (`[Required]`, `[Range]`), що дають відповідь 400 з поясненням.

4. Що таке REST і як у вашому API забезпечується дотримання принципу однорідності інтерфейсу (uniform interface)?

REST (Representational State Transfer) — архітектурний стиль розподілених систем, описаний Роєм Філдингом (2000). Його умови: клієнт-сервер, відсутність стану на сервері між запитами (stateless), кешованість, **однорідний інтерфейс**, багаторівнева система, код на вимогу (необов'язково). Однорідний інтерфейс складається з ідентифікації ресурсів через URI, роботи з ресурсами через їхні представлення, самоописових повідомлень і гіпермедіа (HATEOAS).

У GameStudio.REST цей принцип забезпечено так:

- ресурси названо унікальними іменниками в множині: `/api/projects`, `/api/bugs`, `/api/programmers`, `/api/artists`. Пов'язані ресурси вкладено: `/api/projects/{id}/store-page`, `/bugs`, `/team/{employeeId}`. Дієслів в адресах немає;

- дію визначає HTTP-метод: GET читає, POST створює, PUT оновлює, DELETE видаляє. PUT і DELETE ідемпотентні: повторне додавання учасника в команду знову повертає 204 і нічого не змінює;
- відповіді мають правильні коди (200, 201, 204, 400, 404, 409), заголовки `Content-Type` (`application/json` або `application/problem+json`) і `Location` для нового ресурсу;
- представлення ресурсу — JSON; переліки передаються рядками, помилки — у стандартному форматі ProblemDetails;
- сервер не зберігає стан клієнта між запитами: кожен запит містить усі потрібні дані (ідентифікатор, номер сторінки, тіло).

Гіпермедіа (посилання на пов'язані ресурси в тілі відповіді) повністю не реалізовано — використовується лише заголовок `Location`.

5. Які HTTP-методи використовуються для CRUD-операцій? Наведіть приклади відповідності методів діям у контролерах.

Create — **POST** (на колекцію), Read — **GET** (колекція або один елемент), Update — **PUT** (повна заміна; для часткової зміни є **PATCH**), Delete — **DELETE**. GET безпечний і ідемпотентний, PUT і DELETE ідемпотентні, POST — ні.

HTTP-запит	Дія контролера	Виклик CRUD-сервісу
`GET /api/projects?page=2&amount=5`	`[HttpGet] GetAll(page, amount)`	`ReadAllAsync(page, amount)`
`GET /api/projects/{id}`	`[HttpGet("{id:guid}")] GetById(id)`	`ReadAsync(id)`
`POST /api/projects`	`[HttpPost] Create(ProjectCreateModel)`	`CreateAsync(project)` + `SaveAsync()`
`PUT /api/projects/{id}`	`[HttpPut("{id:guid}")] Update(id, ProjectUpdateModel)`	`ReadAsync` → `UpdateAsync` + `SaveAsync()`
`DELETE /api/projects/{id}`	`[HttpDelete("{id:guid}")] Delete(id)`	`ReadAsync` → `RemoveAsync` + `SaveAsync()`
`PUT /api/projects/{id}/store-page`	`PutStorePage(id, StorePageModel)`	`UpdateAsync(project)` + `SaveAsync()`
`DELETE /api/projects/{id}/team/{employeeId}`	`RemoveTeamMember(id, employeeId)`	`UpdateAsync(project)` + `SaveAsync()`

Так само влаштовано `BugsController`, `ProgrammersController` і `ArtistsController`. Обмеження `{id:guid}` у маршруті гарантує, що до дії потрапить лише коректний Guid.

6. Які HTTP-коди відповідей є коректними для сценаріїв успіху та обробки помилок у вашому API (створення, оновлення, видалення, не знайдено тощо)?

Код	Коли повертається в GameStudio.REST
200 OK	успішний GET; PUT, що повертає оновлений ресурс; PUT сторінки в магазині, якщо вона вже була
201 Created	POST створив ресурс (у тілі — новий ресурс, у заголовку `Location` — його адреса); перше створення сторінки в магазині через PUT
204 No Content	успішний DELETE; додавання учасника до команди (PUT)
400 Bad Request	модель не пройшла перевірку; посилання на неіснуючий проєкт чи програміста; виправлений баг без виконавця; некоректна пагінація
404 Not Found	ресурсу з таким Id немає; у проєкту немає сторінки в магазині; працівник не входить до команди
409 Conflict	запит суперечить стану ресурсу: зміна статусу випущеного (Released) чи скасованого проєкту
500 Internal Server Error	не вдалося зберегти зміни (`SaveAsync` повернув false) або необроблений виняток

ASP.NET Core також автоматично повертає 415 Unsupported Media Type (тіло не в JSON) і 405 Method Not Allowed (метод не підтримується маршрутом). Усі помилки мають формат ProblemDetails.

7. Опишіть інтерфейс ICrudServiceAsync<T>: для чого потрібні асинхронні методи та як вони впливають на продуктивність застосунку?

`ICrudServiceAsync<T>` — узагальнений контракт CRUD-операцій над сутностями будь-якого типу:

- `CreateAsync(T)` додає елемент (false — якщо такий Id уже є);
- `ReadAsync(Guid)` повертає елемент або кидає `KeyNotFoundException`;
- `ReadAllAsync()` повертає всі елементи, `ReadAllAsync(page, amount)` — одну сторінку;
- `UpdateAsync(T)` / `RemoveAsync(T)` реєструють зміну (false — якщо елемента немає);
- `SaveAsync()` зберігає всі зміни однією транзакцією (шаблон «одиниця роботи»).

Методи повертають `Task`, бо звертаються до БД (введення-виведення). Під час `await` (наприклад, `ToListAsync`, `SaveChangesAsync`) потік не блокується, а повертається в пул потоків і обслуговує інші запити. Коли БД відповідає, виконання продовжується. Тому сервер з тією самою кількістю потоків обробляє значно більше одночасних запитів і не вичерпує пул потоків під навантаженням. Окремий запит від цього не стає швидшим (машина станів `async` навіть додає невеликі витрати), але зростає пропускна здатність і масштабованість. `DbContext` не потокобезпечний, тому сервіс впорядковує звернення асинхронним семафором (`SemaphoreSlim.WaitAsync`).

8. Як працює механізм впровадження залежностей (Dependency Injection) в ASP.NET Core і як ви зареєстрували свої сервіси та репозиторії?

ASP.NET Core має вбудований контейнер інверсії керування. Під час запуску в `builder.Services` (IServiceCollection) реєструють пари «абстракція → реалізація» з часом життя:

- **Singleton** — один екземпляр на весь застосунок;
- **Scoped** — один екземпляр на HTTP-запит;
- **Transient** — новий екземпляр щоразу.

Контролер створюється на кожен запит, а його конструктор отримує залежності від контейнера (впровадження через конструктор). Контейнер рекурсивно створює й залежності залежностей.

У GameStudio.REST (код у розділі 1.4) усе зареєстровано як Scoped:

- `AddDbContext<GameStudioContext>` з провайдером SQLite;
- відкриті узагальнені типи `IRepository<>` і `IEntityRepository<>` → `Repository<>`: один рядок обслуговує всі моделі БД;
- перетворювачі `IModelMapper<GameProject, GameProjectModel>` → `GameProjectMapper` тощо;
- `ICrudServiceAsync<GameProject>` → `CrudServiceAsync<GameProject, GameProjectModel>` і так само для `BugReport`, `Employee`, `Programmer`, `Artist`.

Ланцюжок для `ProjectsController`: `ICrudServiceAsync<GameProject>` ← `CrudServiceAsync` ← `Repository<GameProjectModel>` і `GameProjectMapper` ← `GameStudioContext`. Усі вони в межах запиту працюють з одним контекстом. Завдяки DI тести підміняють рядок підключення через `WebApplicationFactory`, не змінюючи код. У GameStudio.MVC контролери так само отримують `GameStudioContext` через конструктор.

9. Як організована пагінація у вашому CRUD-сервісі (ReadAllAsync(int page, int amount)) та для чого вона потрібна?

Сторінки нумеруються з 1, `amount` — розмір сторінки. `CrudServiceAsync` перевіряє параметри (`ArgumentOutOfRangeException`) і викликає `Repository.GetPageAsync`, який будує запит `OrderBy(e => e.Id).Skip((page - 1) * amount).Take(amount)`. EF Core перетворює його на SQL `ORDER BY "Id" LIMIT @amount OFFSET @offset`, тож з БД читаються лише потрібні рядки. Сортування за первинним ключем дає стабільні сторінки.

У контролерах параметри передаються в рядку запиту (`GET /api/projects?page=2&amount=5`; за замовчуванням 1 і 10) і перевіряються атрибутами `[Range(1, int.MaxValue)]` і `[Range(1, 100)]`, а некоректні значення дають 400. Сторінка за межами списку — порожній масив з кодом 200.

Для чого: таблиці містять сотні записів (200 працівників, 159 багів). Повертати їх усі в одній відповіді означало б зайвий трафік, пам'ять і час, а обмеження розміру сторінки захищає сервер від надто великих запитів. У GameStudio.MVC списки теж розбито на сторінки по 15 записів.

10. Яким чином ви тестували роботу свого API, які інструменти або сторінки використовували, та які результати слід включити в PDF-файл для PR?

- **Swagger UI** (`/swagger`) — сторінка, яку генерує Swashbuckle за описом OpenAPI. На ній виконано сценарій із 30 запитів до всіх контролерів, включно з помилковими (розділ 2);
- **автоматичні інтеграційні тести** — NUnit + `WebApplicationFactory<Program>` (пакет Microsoft.AspNetCore.Mvc.Testing): застосунок працює в пам'яті з тимчасовою БД, 12 тестів перевіряють коди й тіла відповідей (розділ 4);
- файл `GameStudio.REST.http` з прикладами запитів для Visual Studio / Rider; як альтернативу можна використати Postman.

У PDF для PR слід включити результати виконання запитів для кожної CRUD-операції кожної сутності: метод, URL, параметри й тіло запиту, код відповіді, тіло та важливі заголовки (`Location`, `Content-Type`). Також потрібні сценарії помилок (400, 404, 409), які показують, що API відповідає вимогам однорідного інтерфейсу.

11. Які кроки потрібно виконати, щоб створити проєкт GameStudio.MVC, та чим відрізняється його структура від REST API?

- створити проєкт за шаблоном «ASP.NET Core Web App (Model-View-Controller)»: `dotnet new mvc -n GameStudio.MVC`, `dotnet sln add GameStudio.MVC`;
- додати посилання на `GameStudio.Infrastructure` (і `GameStudio.Common`), пакети `Microsoft.VisualStudio.Web.CodeGeneration.Design`, `Microsoft.EntityFrameworkCore.Design` / `.Tools`;
- зареєструвати контекст БД у `Program.cs` (`AddDbContext<GameStudioContext>`);
- для кожної моделі запустити генератор «MVC Controller with views, using Entity Framework» (Visual Studio: Add → New Scaffolded Item; або `dotnet aspnet-codegenerator controller ...`, розділ 3);
- доопрацювати згенеровані сторінки та додати пункти меню в `_Layout.cshtml`.

	GameStudio.MVC	**GameStudio.REST**
Контролери	`Controller`, повертають `View()` / `RedirectToAction()`	`ControllerBase` + `[ApiController]`, повертають дані та коди
Інтерфейс	папка `Views`: Razor-представлення .cshtml, `_Layout`, `_ViewImports`, `_ViewStart`	немає; опис API — Swagger
Статичні файли	`wwwroot`: CSS, Bootstrap, jQuery Validation	немає
Маршрутизація	за шаблоном `{controller=Home}/{action=Index}/{id?}`	атрибутами `[Route]`, `[HttpGet]`…
Дані від клієнта	HTML-форми, токен `[ValidateAntiForgeryToken]`, після POST — перенаправлення	JSON у тілі запиту

	GameStudio.MVC	GameStudio.REST
Доступ до даних	згенерований код напряму використовує `GameStudioContext`	через `ICrudServiceAsync<T>` → репозиторій

12. Як ви повторно використали моделі та контекст бази даних з третьої лабораторної у новому MVC/Blazor-застосунку?

Проєкт `GameStudio.MVC` посилається на `GameStudio.Infrastructure` з ЛР3, тому використовує без змін:

- `GameStudioContext` і моделі `GameProjectModel`, `ProgrammerModel`, `ArtistModel`, `BugReportModel`;
- конфігурації Fluent API (зв'язки, `AutoInclude`) і міграції — під час запуску застосунок викликає `Database.MigrateAsync()`;
- `DatabaseLocator`, тому працює з тим самим файлом `Database/gamestudio.db`, що й консольний застосунок та REST API.

Генератор ASP.NET запускався саме для наявних класів (`-m GameStudio.Infrastructure.Models.GameProjectModel -dc GameStudio.Infrastructure.GameStudioContext`). Тому згенерований код працює з `_context.GameProjects`, `_context.Programmers` (набір TPT), `_context.BugReports.Include(b => b.GameProject)`. На сторінці проєкту використано навігаційні властивості ЛР3: `StorePage` (1:1), `Assignments` з `Employee` (N:M), `Bugs` з `Assignee` (1:N).

Українські підписи й формати додано власним `IDisplayMetadataProvider`, а не атрибутами `[Display]` у моделях, тож рівень доступу до даних не змінено. Зміни, зроблені у вебзастосунку, видно через REST API і навпаки, бо обидва працюють з однією БД.